

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM
KHOA ĐIỆN – ĐIỆN TỬ



BÁO CÁO THỰC TẬP AI

BÀI 3: DEEP LEARNING- MULTILAYER PERCEPTION

MÔN HỌC: THỰC HÀNH HỌC MÁY VÀ TRÍ TUỆ NHÂN TẠO

GIẢNG VIÊN HƯỚNG DẪN: PGS_TS. TRƯƠNG NGỌC SƠN

SINH VIÊN THỰC HIỆN: TRƯƠNG PHÚC THỊNH - 23161336

Tp. Hồ Chí Minh, tháng 5 năm 2026

MỤC LỤC

1. GIỚI THIỆU.....	1
1.1 Tổng quan về Deep Learning	1
1.2 Giới thiệu bài toán	1
1.3 Dataset MNIST.....	1
1.4 Mục tiêu bài thực hành	2
2. CƠ SỞ LÝ THUYẾT	3
2.1 Mạng Neural Network (ANN).....	3
2.2 Multilayer Perceptron (MLP)	4
2.2.1 Kiến trúc tổng quát	4
2.2.2 Lan truyền thuận (Forward Propagation)	6
2.2.3 Lan truyền ngược (Backpropagation).....	7
2.3 Activation Functions	9
2.3.1 ReLU (Rectified Linear Unit)	10
2.3.2 Tanh (Hyperbolic Tangent)	11
2.3.3 LeakyReLU	12
2.4 Loss Function – Cross Entropy Loss.....	12
2.5 Optimizer – Adam	15
Bước 1: Tính moment bậc nhất	16
Bước 2: Tính moment bậc hai	16
Bước 3: Hiệu chỉnh bias	16
Bước 4: Cập nhật trọng số	16
2.6 Batch Size	17
2.7 Learning Rate	20
2.8 Dropout và Regularization	22
Khi Dropout quá lớn.....	24
2.9 Confusion Matrix và các chỉ số đánh giá	25
2.9.1 Confusion Matrix.....	25
2.9.2 Accuracy, Precision, Recall, F1-Score	25
3. QUY TRÌNH THỰC HIỆN.....	27

3.1 Chuẩn bị dữ liệu	27
3.2 Xây dựng mô hình	27
3.3 Huấn luyện mô hình	28
3.4 Các tham số sử dụng.....	29
3.5 Công cụ và phần mềm sử dụng	29
4. KẾT QUẢ THỰC NGHIỆM.....	31
4.1 Các bước khởi tạo ban đầu	31
4.1.1 Chuẩn bị môi trường.....	31
4.1.2 Tải dữ liệu MNIST	31
4.1.3 Tạo DataLoader	31
4.2 Bài tập 1: Data Exploration	32
4.2.1 Kích thước train dataset và test dataset	32
Nhận xét.....	32
4.2.2 Hiển thị 10 ảnh đầu tiên của dataset.....	32
Nhận xét.....	33
4.2.3 Nhận xét phân bố dữ liệu.....	33
4.3 Bài tập 2: Phân tích kiến trúc mạng.....	33
4.3.1 Sơ đồ kiến trúc mạng.....	33
4.3.2 Số lượng tham số từng layer.....	36
4.3.3 Tổng số tham số.....	37
Nhận xét.....	37
4.4 Bài tập 3: Thay đổi kiến trúc mạng	38
4.4.1 Kết quả huấn luyện mô hình ban đầu	38
4.4.2 Kết quả huấn luyện mô hình mới và so sánh kết quả.....	39
4.4.3 Bảng so sánh kết quả	40
4.5 Bài tập 4: Thử nghiệm Activation Functions	41
4.5.1 Code thử nghiệm các activation function	41
4.5.2 Kết quả từng activation function	42
4.5.3 Biểu đồ và bảng so sánh accuracy	43
4.6 Bài tập 5: Thử nghiệm Batch Size.....	44
4.6.1 Code thử nghiệm các giá trị batch size.....	44

4.6.2	Kết quả từng giá trị Batch Size.....	45
4.6.3	Biểu đồ và bảng so sánh	46
4.7	Bài tập 6: Thử nghiệm Learning Rate	47
4.7.1	Code thử nghiệm các giá trị learning rate	47
4.7.2	Kết quả từng giá trị Learning Rate	48
4.7.3	Biểu đồ Loss theo learning rate	49
4.7.4	Bảng so sánh kết quả	50
	Nhận xét.....	51
4.8	Bài tập 7: Dropout	51
4.8.1	Code mô hình có Dropout	51
4.8.2	Kết quả trước và sau Dropout.....	52
4.8.3	Bảng so sánh accuracy và overfitting.....	52
4.9	Bài tập 8: Thử nghiệm Training Epochs	53
4.9.1	Code thử nghiệm số epochs.....	53
4.9.2	Kết quả từng số epoch	54
4.9.3	Biểu đồ Training Loss và Test Accuracy theo Epoch	55
4.9.4	Bảng so sánh kết quả	56
4.10	Bài tập 9: Confusion Matrix	57
4.10.1	Code tính Confusion Matrix.....	57
4.10.2	Heatmap Confusion Matrix	58
4.11	Bài tập 10: Cải thiện mô hình đạt accuracy > 97%	59
4.11.1	Chiến lược cải thiện.....	59
4.11.2	Code mô hình tối ưu	59
4.11.3	Kết quả đạt được.....	60
5.	THẢO LUẬN VÀ ĐÁNH GIÁ	61
5.1	Ảnh hưởng của kiến trúc mạng đến hiệu suất	61
5.2	Vai trò của Activation Function	61
5.3	Ảnh hưởng của Batch Size	61
5.4	Ảnh hưởng của Learning Rate đến quá trình hội tụ	62
5.5	Hiệu quả của Dropout trong việc giảm Overfitting.....	62
5.6	Ảnh hưởng của số Epoch.....	63

5.7 Phân tích Confusion Matrix.....	63
5.8 Tổng hợp cấu hình tốt nhất.....	64
5.9 Hạn chế và hướng cải thiện	64
6. KẾT LUẬN	65
6.1 Tóm tắt kết quả đạt được	65
6.2 Bài học rút ra	65
6.3 Hướng phát triển tiếp theo	65

DANH MỤC HÌNH ẢNH

Hình 1: Kiến trúc mạng Multilayer Perceptron (MLP) cho bài toán phân loại chữ số viết tay MNIST	6
Hình 2: Minh họa quá trình Forward Propagation trong mạng neural cho minh hình minh họa.....	7
Hình 3: Minh họa quá trình Backpropagation trong mạng neural đơn giản	9
Hình 4: Minh họa quá trình Backpropagation trong mạng neural đơn giản	10
Hình 6: So sánh Batch Size nhỏ và Batch Size lớn trong quá trình cập nhật gradient	20
Hình 7: Ảnh hưởng của Learning Rate đến quá trình tối ưu và hội tụ của mô hình	22
Hình 8: Minh họa cơ chế Dropout với các neuron được vô hiệu hóa ngẫu nhiên trong quá trình huấn luyện	25
Hình 9: Dataset MNIST được tải thành công.....	31
Hình 10: Kết quả in kích thước dataset (train: 60000, test: 10000)	32
Hình 11: Hiển thị 10 ảnh đầu tiên của dataset MNIST	33
Hình 12: Kiến trúc mô hình MLP	34
Hình 14: Kết quả in số lượng tham số từng layer.....	37
Hình 15: Kết quả huấn luyện mô hình ban đầu	38
Hình 16: Biểu đồ training loss.....	38
Hình 17: Kết quả huấn luyện mô hình mới	40
Hình 18: Kết quả huấn luyện với ReLU	42
Hình 19: Kết quả huấn luyện với Tanh	42
Hình 20: Kết quả huấn luyện với LeakyReLU	43
Hình 21: Kết quả huấn luyện với Batch Size = 32	45
Hình 22: Kết quả huấn luyện với Batch Size = 64	45
Hình 23: Kết quả huấn luyện với Batch Size = 128	46
Hình 24: So sánh accuracy và thời gian huấn luyện theo batch size.....	46
Hình 25: Kết quả huấn luyện với lr = 0.1	48
Hình 26: Kết quả huấn luyện với lr = 0.01	48
Hình 27: Kết quả huấn luyện với lr = 0.001	49
Hình 28: Kết quả huấn luyện với lr = 0.0001	49
Hình 29: Biểu đồ loss theo Learning Rate.....	50

Hình 30: Accuracy theo từng learning rate	50
Hình 31: Kết quả mô hình dùng Dropout ($p=0.5$)	52
Hình 32: Kết quả huấn luyện với 5 Epochs.....	54
Hình 33: Kết quả huấn luyện với 10 Epochs.....	54
Hình 34: Kết quả huấn luyện với 20 Epochs.....	55
Hình 35: Biểu đồ Training Loss theo Epoch (5, 10, 20 epochs)	55
Hình 36: Biểu đồ Test Accuracy theo Epoch	56
Hình 37: Heatmap Confusion Matrix của mô hình MLP	58
Hình 38: Kết quả huấn luyện mô hình tối ưu	60

1. GIỚI THIỆU

1.1 Tổng quan về Deep Learning

Deep Learning là một nhánh của Machine Learning, sử dụng mạng neural network nhiều lớp (multi-layer neural network) để tự động học các đặc trưng từ dữ liệu. Khác với các phương pháp Machine Learning truyền thống yêu cầu trích xuất đặc trưng thủ công, Deep Learning có khả năng học trực tiếp từ dữ liệu thô, từ đó xây dựng các hệ thống thông minh trong các lĩnh vực như nhận dạng ảnh, nhận dạng giọng nói và xử lý ngôn ngữ tự nhiên.

Trong những năm gần đây, Deep Learning đã đạt được nhiều bước tiến vượt bậc nhờ sự phát triển của dữ liệu lớn (Big Data), khả năng tính toán mạnh mẽ của GPU và các thuật toán tối ưu hóa hiện đại. Trong đó, mô hình Multilayer Perceptron (MLP) là một trong những kiến trúc neural network cơ bản và quan trọng, đặt nền tảng cho sự phát triển của các kiến trúc nâng cao như CNN, RNN và Transformer.

1.2 Giới thiệu bài toán

Bài toán nhận dạng chữ số viết tay là một trong những bài toán kinh điển trong lĩnh vực Computer Vision và Machine Learning. Đây là bài toán phân loại ảnh đầu vào thành một trong 10 lớp tương ứng với các chữ số từ 0 đến 9.

Ứng dụng thực tế của bài toán này rất phong phú: nhận dạng số tài khoản trên séc ngân hàng, đọc bưu kiện tự động tại bưu điện, nhận dạng số trên biển số xe, hay số hóa tài liệu viết tay trong các hệ thống lưu trữ.

1.3 Dataset MNIST

Dataset MNIST (Modified National Institute of Standards and Technology) là bộ dữ liệu chuẩn được sử dụng rộng rãi trong nghiên cứu Machine Learning. Dataset bao gồm:

- 70.000 ảnh chữ số viết tay (60.000 ảnh huấn luyện, 10.000 ảnh kiểm tra)
- Kích thước mỗi ảnh: 28×28 pixels, thang xám (grayscale)
- 10 lớp phân loại: các chữ số từ 0 đến 9
- Mỗi ảnh được biểu diễn bằng 784 đặc trưng ($28 \times 28 = 784$ pixels)

Dữ liệu MNIST có đặc điểm tương đối cân bằng giữa các lớp, phù hợp để huấn luyện mô hình phân loại đa lớp với MLP.

1.4 Mục tiêu bài thực hành

Bài thực hành này nhằm đạt các mục tiêu sau:

- Hiểu kiến trúc và nguyên lý hoạt động của Multilayer Perceptron (MLP)
- Xây dựng và huấn luyện mô hình MLP bằng thư viện PyTorch
- Đánh giá mô hình thông qua accuracy và confusion matrix
- Khảo sát ảnh hưởng của các siêu tham số: kiến trúc mạng, activation function, batch size, learning rate, dropout, số epoch
- Tối ưu hóa mô hình để đạt accuracy > 97% trên tập test

2. CƠ SỞ LÝ THUYẾT

2.1 Mạng Neural Network (ANN)

Mạng Neural Network nhân tạo (Artificial Neural Network – ANN) là mô hình tính toán được phát triển dựa trên nguyên lý hoạt động của hệ thần kinh sinh học trong não người. ANN bao gồm nhiều neuron nhân tạo liên kết với nhau thành từng lớp nhằm xử lý và học từ dữ liệu đầu vào.

Một mạng neural cơ bản thường gồm ba thành phần chính:

- **Input Layer (lớp đầu vào):** nhận dữ liệu từ bên ngoài
- **Hidden Layer (lớp ẩn):** thực hiện quá trình học và trích xuất đặc trưng
- **Output Layer (lớp đầu ra):** đưa ra kết quả dự đoán cuối cùng

Mỗi neuron trong mạng neural nhận các giá trị đầu vào, sau đó thực hiện phép tính tổng có trọng số theo công thức:

$$z = \sum_{i=1}^n w_i x_i + b$$

Trong đó:

- (x_i) : giá trị đầu vào
- (w_i) : trọng số của từng đầu vào
- (b) : độ lệch (bias)
- (z) : giá trị tổng hợp trước khi qua hàm kích hoạt

Sau khi tính được giá trị (z) , neuron sẽ áp dụng hàm kích hoạt (Activation Function) để tạo ra đầu ra:

$$a = f(z)$$

Hàm kích hoạt đóng vai trò tạo tính phi tuyến cho mô hình, giúp mạng neural có khả năng học được các đặc trưng phức tạp của dữ liệu. Một số hàm kích hoạt phổ biến gồm:

- ReLU (Rectified Linear Unit)
- Sigmoid
- Tanh
- LeakyReLU

Trong bài thực hành này, hàm ReLU được sử dụng vì có khả năng hội tụ nhanh và giảm hiện tượng vanishing gradient trong quá trình huấn luyện.

Quá trình học của mạng neural được thực hiện thông qua thuật toán Backpropagation kết hợp với thuật toán tối ưu hóa (optimizer) như Adam hoặc SGD. Trong quá trình huấn luyện:

1. Dữ liệu được đưa vào mạng để tạo dự đoán
2. Tính sai số giữa kết quả dự đoán và nhãn thực tế thông qua hàm mất mát (Loss Function)
3. Lan truyền ngược sai số để cập nhật trọng số
4. Lặp lại nhiều lần nhằm tối ưu mô hình

Mục tiêu của quá trình huấn luyện là tìm ra bộ trọng số tối ưu để mô hình đạt độ chính xác cao nhất trên tập dữ liệu kiểm tra.

2.2 Multilayer Perceptron (MLP)

2.2.1 Kiến trúc tổng quát

Multilayer Perceptron (MLP) là một dạng mạng neural network truyền thẳng (Feedforward Neural Network), trong đó các neuron giữa các lớp được kết nối đầy đủ với nhau (fully connected). Dữ liệu được truyền theo một chiều từ lớp đầu vào đến lớp đầu ra mà không tồn tại vòng lặp ngược.

Một mô hình MLP cơ bản bao gồm ba thành phần chính:

- **Input Layer (lớp đầu vào):** nhận dữ liệu đầu vào của mô hình. Đối với dataset MNIST, mỗi ảnh có kích thước 28×28 pixels nên được biểu diễn bởi 784 giá trị đầu vào. Do đó, input layer gồm 784 neuron tương ứng với 784 pixels của ảnh.
- **Hidden Layers (các lớp ẩn):** thực hiện quá trình học đặc trưng của dữ liệu. Mỗi neuron trong hidden layer nhận toàn bộ đầu ra từ lớp trước đó, sau đó thực hiện phép tính tuyến tính kết hợp với hàm kích hoạt phi tuyến để học các đặc trưng phức tạp hơn.
- **Output Layer (lớp đầu ra):** đưa ra kết quả phân loại cuối cùng. Với bài toán nhận dạng chữ số MNIST gồm 10 lớp, output layer có 10 neuron tương ứng với các chữ số từ 0 đến 9.

Trong bài thực hành này, mô hình MLP được xây dựng với kiến trúc:

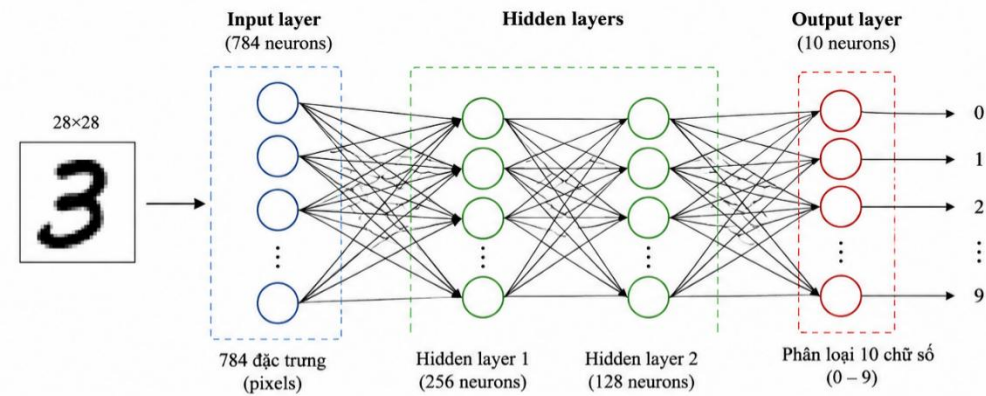
$$784 \rightarrow 256 \rightarrow 128 \rightarrow 10$$

Trong đó:

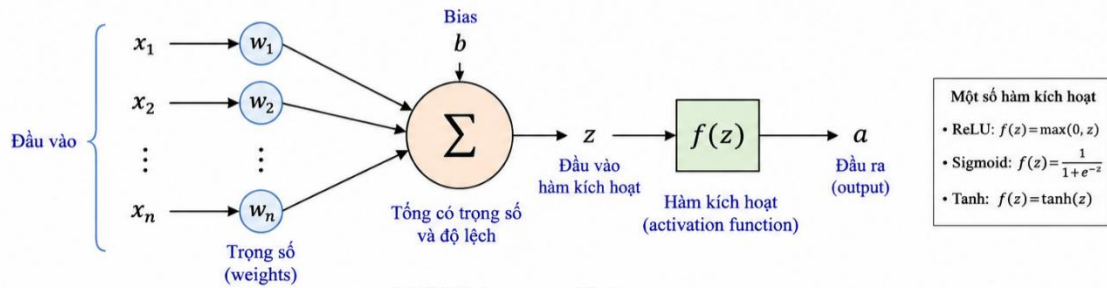
- 784 neuron đầu vào biểu diễn dữ liệu ảnh
- Hai hidden layer gồm 256 và 128 neuron
- 10 neuron đầu ra biểu diễn xác suất của từng lớp chữ số

Các hidden layer đóng vai trò quan trọng trong việc học và trích xuất đặc trưng từ dữ liệu. Khi số lượng hidden layer và neuron tăng lên, khả năng biểu diễn dữ liệu của mô hình cũng mạnh hơn. Tuy nhiên, điều này đồng thời làm tăng độ phức tạp tính toán và nguy cơ overfitting nếu mô hình học quá sát dữ liệu huấn luyện.

Ví dụ trong bài toán MNIST:



(a) Kiến trúc mạng Multilayer Perceptron (MLP) cho bài toán MNIST



(b) Mô hình neuron nhân tạo

Hình 1: Kiến trúc mạng Multilayer Perceptron (MLP) cho bài toán phân loại chữ số viết tay MNIST

2.2.2 Lan truyền thuận (Forward Propagation)

Lan truyền thuận (Forward Propagation) là quá trình dữ liệu được truyền từ input layer qua các hidden layer đến output layer để tạo ra dự đoán cuối cùng của mô hình.

Tại mỗi lớp thứ (l) , đầu ra được tính theo công thức:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

Sau đó áp dụng hàm kích hoạt:

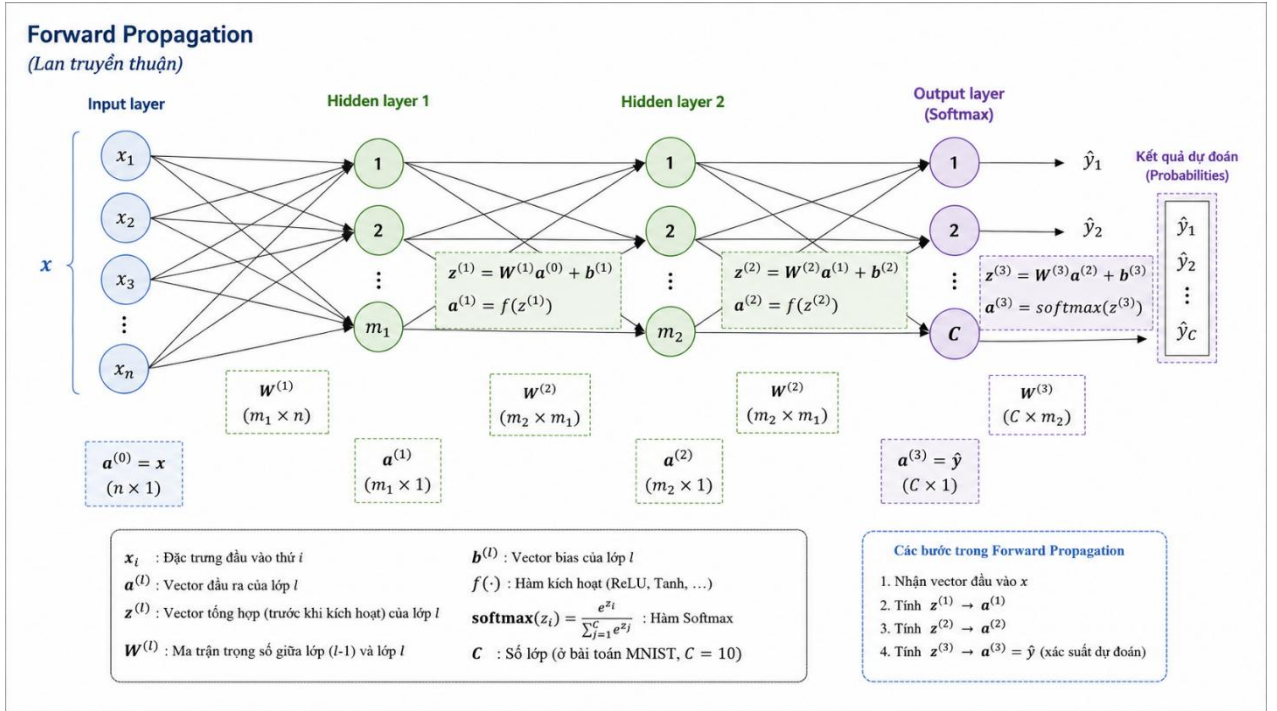
$$a^{(l)} = f(z^{(l)})$$

Trong đó:

- $(W^{(l)})$: ma trận trọng số của lớp (l)
- $(b^{(l)})$: vector bias

- $(a^{(l-1)})$: đầu ra của lớp trước
- (f) : hàm kích hoạt
- $(a^{(l)})$: đầu ra của lớp hiện tại

Quá trình lan truyền thuận giúp mô hình chuyển đổi dữ liệu đầu vào thành các đặc trưng có ý nghĩa hơn qua từng lớp ẩn. Ở lớp cuối cùng, mô hình đưa ra xác suất dự đoán cho từng lớp thông qua hàm Softmax.



Hình 2: Minh họa quá trình Forward Propagation trong mạng neural cho minh hình minh họa

2.2.3 Lan truyền ngược (Backpropagation)

Backpropagation là thuật toán cốt lõi trong quá trình huấn luyện mạng neural network. Nếu lan truyền thuận dùng để đưa dữ liệu đầu vào qua các lớp của mạng và tạo ra kết quả dự đoán, thì lan truyền ngược được sử dụng để tính toán sai số và cập nhật các trọng số nhằm cải thiện mô hình.

Sau khi mô hình thực hiện lan truyền thuận, kết quả dự đoán được so sánh với nhãn thực tế thông qua hàm mất mát (Loss Function). Giá trị loss cho biết mức độ sai lệch giữa đầu ra

dự đoán và kết quả đúng. Loss càng lớn nghĩa là mô hình dự đoán càng sai, ngược lại loss càng nhỏ thì mô hình dự đoán càng chính xác.

Mục tiêu của quá trình huấn luyện là làm giảm giá trị loss qua từng lần lặp. Để thực hiện điều này, Backpropagation tính gradient của hàm mất mát đối với từng tham số trong mạng, bao gồm trọng số (weights) và độ lệch (bias). Gradient cho biết nếu một trọng số thay đổi thì giá trị loss sẽ thay đổi như thế nào.

Việc tính gradient được thực hiện từ lớp đầu ra quay ngược về các lớp trước đó bằng quy tắc đạo hàm dây chuyền (Chain Rule). Nhờ đó, mạng có thể xác định mức độ đóng góp của từng neuron và từng trọng số vào sai số cuối cùng.

Sau khi có gradient, các trọng số được cập nhật theo hướng ngược chiều gradient để giảm hàm mất mát:

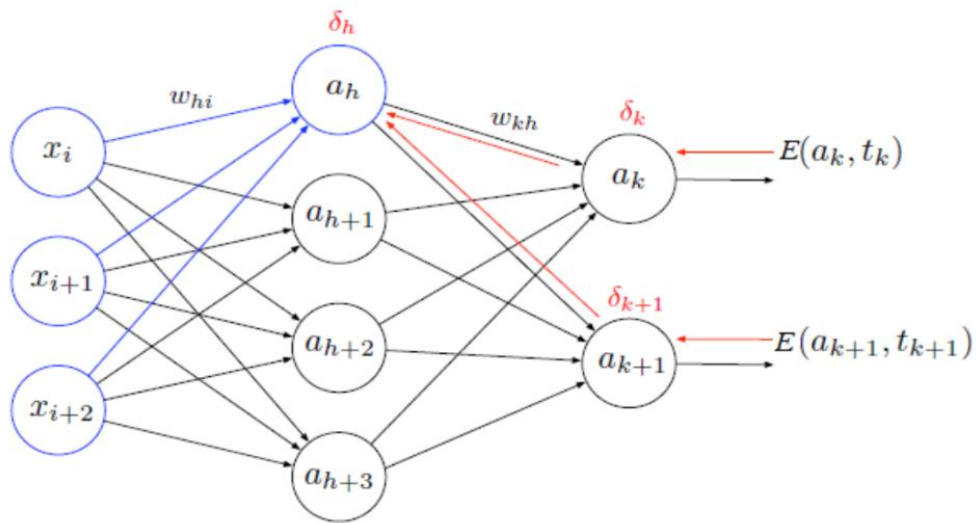
$$w = w - \eta \frac{\partial L}{\partial w}$$

Trong đó:

- w : trọng số cần cập nhật
- η : learning rate, quyết định mức độ thay đổi của trọng số sau mỗi lần cập nhật
- L : hàm mất mát
- $\left(\frac{\partial L}{\partial w}\right)$: gradient của hàm mất mát theo trọng số (w)

Nếu gradient dương, trọng số sẽ được giảm xuống. Nếu gradient âm, trọng số sẽ được tăng lên. Nhờ vậy, mô hình dần di chuyển đến vùng có giá trị loss nhỏ hơn.

Quá trình Backpropagation thường được kết hợp với các thuật toán tối ưu như SGD hoặc Adam. Trong bài thực hành này, Adam optimizer được sử dụng để tự động điều chỉnh tốc độ cập nhật trọng số, giúp quá trình huấn luyện ổn định và hội tụ nhanh hơn.



Hình 3: Minh họa quá trình Backpropagation trong mạng neural đơn giản

Toàn bộ quá trình học của mạng neural có thể tóm tắt như sau:

1. Đưa dữ liệu đầu vào vào mô hình.
2. Thực hiện lan truyền thuận để tạo kết quả dự đoán.
3. Tính loss giữa kết quả dự đoán và nhãn thực tế.
4. Sử dụng Backpropagation để tính gradient.
5. Cập nhật trọng số bằng optimizer.
6. Lặp lại quá trình trên qua nhiều epoch cho đến khi mô hình đạt độ chính xác mong muốn.

Như vậy, Backpropagation giúp mạng neural tự động học từ dữ liệu bằng cách liên tục điều chỉnh các trọng số. Đây là cơ chế quan trọng giúp mô hình MLP cải thiện khả năng phân loại chữ số viết tay trong bài toán MNIST.

2.3 Activation Functions

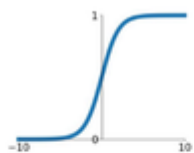
Hàm kích hoạt (Activation Function) là thành phần quan trọng trong mạng neural network, giúp đưa tính phi tuyến vào mô hình. Nếu không sử dụng hàm kích hoạt, toàn bộ mạng neural chỉ tương đương một phép biến đổi tuyến tính đơn giản và không thể học được các mối quan hệ phức tạp trong dữ liệu.

Trong bài thực hành này, ba hàm kích hoạt phổ biến được sử dụng gồm ReLU, Tanh và LeakyReLU nhằm khảo sát ảnh hưởng của chúng đến hiệu suất mô hình.

Activation Functions

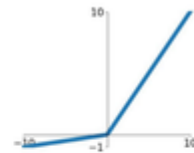
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



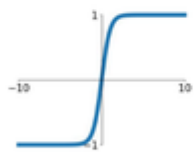
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

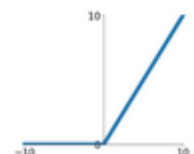


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

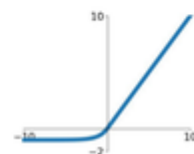
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Hình 4: Minh họa quá trình Activation trong mạng neural đơn giản

2.3.1 ReLU (Rectified Linear Unit)

ReLU là hàm kích hoạt được sử dụng phổ biến nhất trong các mô hình Deep Learning hiện đại nhờ cấu trúc đơn giản và hiệu quả tính toán cao.

Hàm được định nghĩa:

$$f(x) = \max(0, x)$$

Đặc điểm của ReLU:

- Khi ($x < 0$), đầu ra bằng 0
- Khi ($x > 0$), đầu ra bằng chính giá trị đầu vào

Điều này giúp ReLU giảm đáng kể hiện tượng vanishing gradient so với các hàm kích hoạt truyền thống như Sigmoid hoặc Tanh.

Ưu điểm của ReLU:

- Tính toán đơn giản và nhanh
- Giúp mô hình hội tụ nhanh hơn
- Hoạt động hiệu quả với mạng neural sâu

Tuy nhiên, ReLU có thể gặp hiện tượng “dying ReLU”. Khi đầu vào luôn âm, neuron sẽ luôn cho đầu ra bằng 0 và gradient bằng 0, khiến neuron không còn được cập nhật trong quá trình huấn luyện.

Trong bài thực hành này, ReLU cho khả năng huấn luyện ổn định và thường đạt accuracy cao nhất trên tập dữ liệu MNIST.

2.3.2 Tanh (Hyperbolic Tangent)

Tanh là hàm kích hoạt phi tuyến có dạng:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Hàm Tanh cho đầu ra trong khoảng:

$(-1,1)$

Khác với Sigmoid, đầu ra của Tanh đối xứng qua gốc tọa độ, giúp dữ liệu được chuẩn hóa quanh giá trị 0. Điều này thường giúp gradient ổn định hơn trong quá trình huấn luyện.

Ưu điểm của Tanh:

- Có tính phi tuyến mạnh
- Đầu ra đối xứng quanh 0
- Gradient thường lớn hơn Sigmoid

Nhược điểm:

- Khi giá trị đầu vào quá lớn hoặc quá nhỏ, gradient tiến gần về 0
- Có thể xảy ra hiện tượng vanishing gradient khi mạng quá sâu

Trong bài thực hành, Tanh cho kết quả ổn định nhưng tốc độ hội tụ thường chậm hơn ReLU.

2.3.3 LeakyReLU

LeakyReLU là phiên bản cải tiến của ReLU nhằm khắc phục hiện tượng dying ReLU.

Hàm được định nghĩa:

$$f(x) = \max(\alpha x, x)$$

Trong đó:

- α : là hằng số nhỏ, thường chọn bằng 0.01

Khác với ReLU, LeakyReLU vẫn cho phép một gradient nhỏ truyền qua khi đầu vào âm. Điều này giúp neuron tiếp tục được cập nhật thay vì hoàn toàn “chết”.

Ưu điểm của LeakyReLU:

- Giảm hiện tượng dying ReLU
- Giữ được tốc độ hội tụ nhanh
- Gradient ổn định hơn ở vùng âm

Nhược điểm:

- Cần lựa chọn hệ số α phù hợp
- Đôi khi không cải thiện đáng kể so với ReLU trong các bài toán đơn giản

Trong bài thực hành này, LeakyReLU cho kết quả khá tốt và ổn định trong quá trình huấn luyện mô hình MLP.

2.4 Loss Function – Cross Entropy Loss

Trong các bài toán phân loại đa lớp (multi-class classification), hàm mất mát đóng vai trò đánh giá mức độ sai lệch giữa kết quả dự đoán của mô hình và nhãn thực tế. Một mô hình có giá trị loss càng nhỏ thì khả năng dự đoán càng chính xác.

Trong bài thực hành này, hàm mất mát Cross Entropy Loss được sử dụng để huấn luyện mô hình MLP trên dataset MNIST. Đây là hàm mất mát phổ biến và hiệu quả nhất cho các bài toán phân loại nhiều lớp trong Deep Learning.

Cross Entropy Loss đo mức độ khác biệt giữa phân phối xác suất dự đoán của mô hình và phân phối xác suất thực tế của dữ liệu.

Với:

- (N): số lượng mẫu dữ liệu
- (C): số lớp phân loại

Hàm Cross Entropy Loss được định nghĩa như sau:

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(p_{i,c})$$

Trong đó:

- $(y_{i,c})$: nhãn thực tế của mẫu thứ (i) tại lớp (c)
- $(p_{i,c})$: xác suất dự đoán của mô hình cho lớp (c)
- (L): giá trị hàm mất mát

Trong bài toán phân loại, nhãn thực tế thường được biểu diễn dưới dạng one-hot vector. Ví dụ với bài toán MNIST gồm 10 lớp. Nếu ảnh là chữ số 3 thì vector nhãn sẽ là: [0,0,0,1,0,0,0,0,0,0]

Sau khi dữ liệu đi qua output layer, mô hình sử dụng hàm Softmax để chuyển đổi đầu ra thành xác suất dự đoán cho từng lớp:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

Hàm Softmax đảm bảo:

- Tổng các xác suất bằng 1
- Lớp có xác suất lớn nhất sẽ là kết quả dự đoán cuối cùng

Cross Entropy Loss hoạt động bằng cách:

- tăng loss khi mô hình dự đoán sai
- giảm loss khi mô hình dự đoán đúng với xác suất cao

Ví dụ:

- Nếu mô hình dự đoán đúng lớp với xác suất 0.99 thì loss rất nhỏ
- Nếu mô hình dự đoán sai hoặc xác suất lớp đúng thấp thì loss sẽ lớn

Do đó, quá trình huấn luyện sẽ cố gắng cập nhật trọng số nhằm giảm giá trị loss qua từng epoch.

Cross Entropy Loss thường được kết hợp với Softmax activation ở output layer vì:

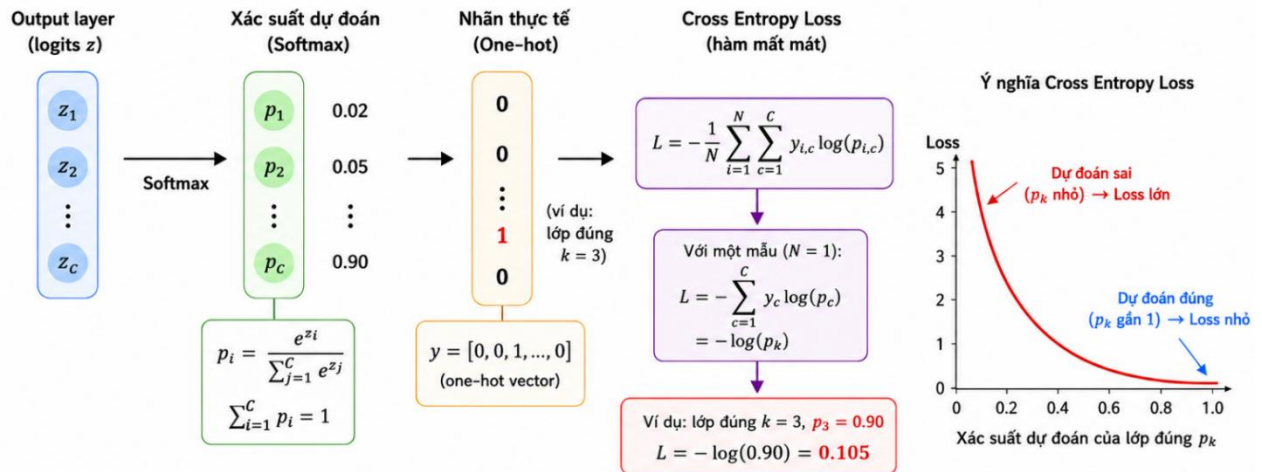
- Softmax chuyển đầu ra thành phân phối xác suất
- Cross Entropy đánh giá độ sai khác giữa xác suất dự đoán và nhãn thực tế

Sự kết hợp này giúp quá trình huấn luyện ổn định, gradient rõ ràng và mô hình hội tụ nhanh hơn.

Trong PyTorch, Cross Entropy Loss được sử dụng thông qua:

```
criterion = nn.CrossEntropyLoss()
```

Hàm này đã tích hợp cả Softmax và Cross Entropy trong cùng một phép tính, giúp tối ưu hiệu năng và tránh sai số số học trong quá trình huấn luyện.



Hình 5: Minh họa Softmax và Cross Entropy Loss trong mạng neural

2.5 Optimizer – Adam

Trong quá trình huấn luyện mạng neural network, sau khi thuật toán Backpropagation tính được gradient của hàm mất mát, mô hình cần một phương pháp để cập nhật các trọng số nhằm giảm sai số dự đoán. Quá trình này được thực hiện bởi thuật toán tối ưu hóa (Optimizer).

Trong bài thực hành này, thuật toán Adam (Adaptive Moment Estimation) được sử dụng để tối ưu mô hình MLP. Adam là một trong những optimizer phổ biến nhất trong Deep Learning nhờ khả năng hội tụ nhanh, ổn định và hoạt động hiệu quả trên nhiều loại dữ liệu khác nhau.

Adam được phát triển dựa trên sự kết hợp ưu điểm của:

- AdaGrad: thích nghi learning rate cho từng tham số
- RMSProp: giảm dao động gradient và cải thiện tốc độ hội tụ

Khác với SGD (Stochastic Gradient Descent) chỉ sử dụng gradient hiện tại để cập nhật trọng số, Adam sử dụng:

- moment bậc nhất (First Moment): trung bình động của gradient
- moment bậc hai (Second Moment): trung bình động của bình phương gradient

Điều này giúp Adam:

- cập nhật trọng số ổn định hơn
- giảm dao động trong quá trình học
- hội tụ nhanh hơn SGD truyền thống

Quá trình cập nhật của Adam gồm các bước:

Bước 1: Tính moment bậc nhất

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Bước 2: Tính moment bậc hai

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Trong đó:

- (g_t) : gradient tại bước cập nhật thứ (t)
- (m_t) : moment bậc nhất
- (v_t) : moment bậc hai
- (β_1, β_2) : hệ số suy giảm

Do (m_t) và (v_t) ban đầu có xu hướng lệch về 0, Adam sử dụng thêm bước hiệu chỉnh bias:

Bước 3: Hiệu chỉnh bias

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\widehat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Bước 4: Cập nhật trọng số

$$\theta_t = \theta_{t-1} - \frac{\alpha \widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}$$

Trong đó:

- (θ_t) : trọng số tại bước (t)
- (α) : learning rate
- (ϵ) : hằng số nhỏ giúp tránh chia cho 0

Các siêu tham số mặc định của Adam thường được sử dụng:

- $(\beta_1 = 0.9)$
- $(\beta_2 = 0.999)$
- $(\epsilon = 10^{-8})$

Ưu điểm của Adam:

- Hội tụ nhanh
- Hoạt động ổn định trên dữ liệu lớn
- Tự động điều chỉnh learning rate cho từng tham số
- Hiệu quả với mạng neural sâu

Nhược điểm:

- Tốn bộ nhớ hơn SGD
- Đôi khi tổng quát hóa (generalization) kém hơn SGD trên một số bài toán lớn

Trong bài thực hành này, Adam giúp mô hình MLP huấn luyện ổn định và đạt accuracy cao trên tập dữ liệu MNIST.

Trong PyTorch, Adam optimizer được sử dụng như sau:

```
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

2.6 Batch Size

Trong quá trình huấn luyện mạng neural network, dữ liệu huấn luyện thường không được đưa toàn bộ vào mô hình cùng một lúc mà được chia thành nhiều nhóm nhỏ gọi là batch. Batch size là số lượng mẫu dữ liệu được xử lý trong một lần cập nhật trọng số của mô hình.

Ví dụ:

- Batch size = 32 nghĩa là mô hình sẽ xử lý 32 ảnh trước khi thực hiện cập nhật trọng số một lần.
- Nếu tập huấn luyện MNIST có 60.000 ảnh và batch size = 64, số lần cập nhật trọng số trong một epoch sẽ là:

$$\frac{60000}{64} \approx 937$$

Batch size ảnh hưởng trực tiếp đến:

- tốc độ huấn luyện
- độ ổn định của gradient
- khả năng hội tụ của mô hình
- mức sử dụng bộ nhớ GPU/CPU

Batch Size nhỏ

Khi batch size nhỏ (ví dụ: 32):

- Mô hình cập nhật trọng số thường xuyên hơn
- Gradient có độ dao động lớn hơn do chỉ dựa trên một lượng dữ liệu nhỏ
- Giúp mô hình dễ thoát khỏi local minima
- Khả năng tổng quát hóa (generalization) thường tốt hơn

Tuy nhiên:

- Tốc độ huấn luyện chậm hơn
- Thời gian training kéo dài
- GPU hoạt động chưa tối ưu

Batch size nhỏ thường phù hợp với:

- dữ liệu phức tạp

- mô hình lớn
- hệ thống có bộ nhớ hạn chế

Batch Size lớn

Khi batch size lớn (ví dụ: 128):

- Gradient ổn định hơn do được tính trên nhiều mẫu dữ liệu
- Tận dụng tốt khả năng xử lý song song của GPU
- Tốc độ huấn luyện nhanh hơn

Tuy nhiên:

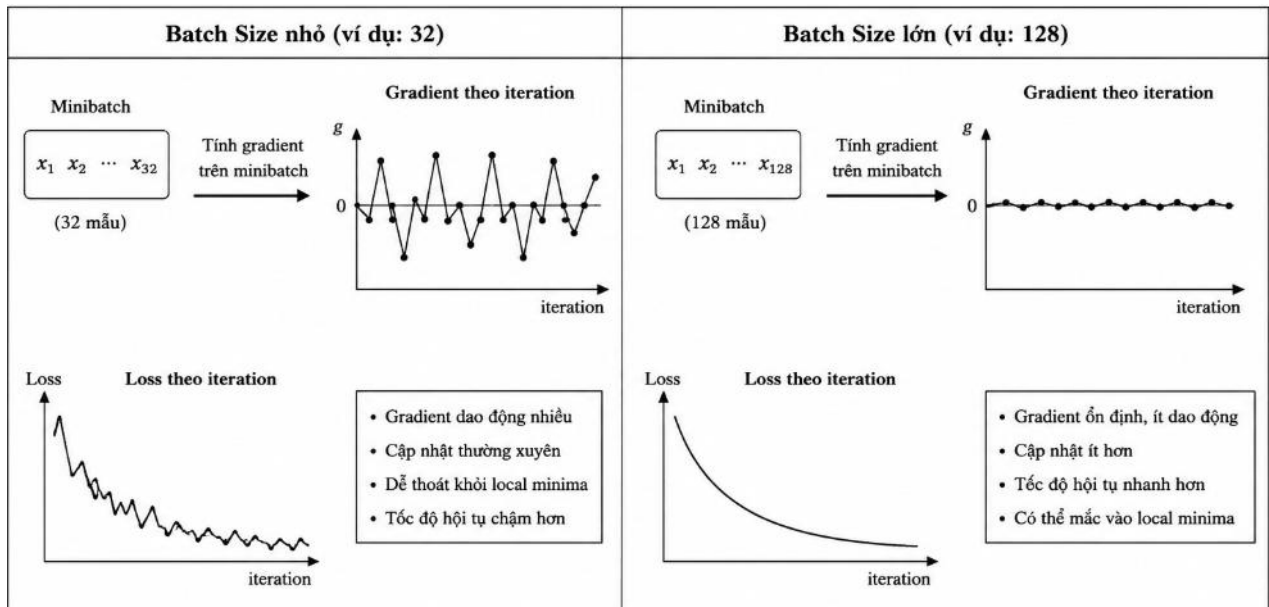
- Mô hình có thể bị mắc tại local minima hoặc saddle point
- Khả năng tổng quát hóa đôi khi giảm
- Yêu cầu nhiều bộ nhớ hơn

Batch size quá lớn cũng có thể làm mô hình hội tụ kém linh hoạt vì gradient thay đổi ít qua từng lần cập nhật.

Ngoài ra, batch size còn ảnh hưởng đến số lần cập nhật trọng số trong mỗi epoch:

- Batch size nhỏ → nhiều lần cập nhật hơn
- Batch size lớn → ít lần cập nhật hơn

Điều này ảnh hưởng trực tiếp đến quá trình hội tụ của mô hình trong huấn luyện Deep Learning.



Hình 6: So sánh Batch Size nhỏ và Batch Size lớn trong quá trình cập nhật gradient

2.7 Learning Rate

Learning Rate (ký hiệu: η hoặc α) là một siêu tham số quan trọng trong quá trình huấn luyện mạng neural network. Learning rate quyết định mức độ thay đổi của trọng số sau mỗi lần cập nhật gradient trong quá trình tối ưu hóa.

Trong thuật toán Gradient Descent, trọng số được cập nhật theo công thức:

$$w = w - \eta \frac{\partial L}{\partial w}$$

Trong đó:

- w : trọng số của mô hình
- η : learning rate
- $\left(\frac{\partial L}{\partial w}\right)$: gradient của hàm mất mát theo trọng số

Learning rate ảnh hưởng trực tiếp đến:

- tốc độ hội tụ của mô hình

- độ ổn định trong huấn luyện
- khả năng đạt được nghiệm tối ưu

Learning Rate quá lớn

Khi learning rate quá lớn:

- Trọng số thay đổi quá mạnh sau mỗi lần cập nhật
- Mô hình có thể vượt qua điểm tối ưu
- Loss dao động mạnh hoặc tăng liên tục
- Quá trình huấn luyện dễ bị phân kỳ (divergence)

Điều này khiến mô hình:

- không hội tụ
- accuracy thấp
- huấn luyện không ổn định

Ví dụ: learning rate = 0.1 thường làm loss giảm rất nhanh ở giai đoạn đầu nhưng dễ dao động mạnh quanh điểm tối ưu.

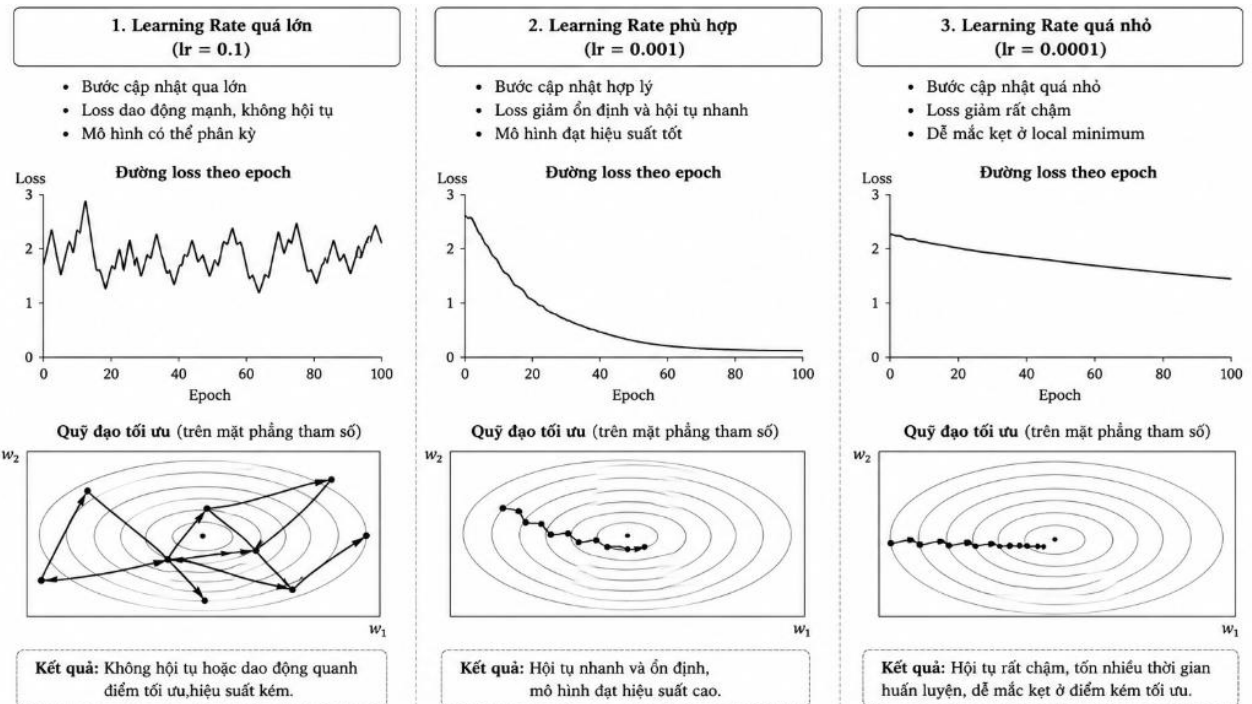
Learning Rate quá nhỏ

Khi learning rate quá nhỏ:

- Trọng số thay đổi rất ít qua mỗi iteration
- Quá trình huấn luyện diễn ra chậm
- Mô hình cần nhiều epoch hơn để hội tụ
- Có thể bị mắc tại local minimum hoặc saddle point

Mặc dù huấn luyện ổn định hơn, nhưng thời gian training sẽ tăng đáng kể.

Ví dụ: learning rate = 0.0001 thường giúp loss giảm ổn định nhưng tốc độ hội tụ rất chậm.



Hình 7: Ảnh hưởng của Learning Rate đến quá trình tối ưu và hội tụ của mô hình

2.8 Dropout và Regularization

Trong Deep Learning, một trong những vấn đề phổ biến nhất là hiện tượng overfitting. Overfitting xảy ra khi mô hình học quá chi tiết trên tập huấn luyện, bao gồm cả nhiễu và các đặc trưng không quan trọng. Khi đó, mô hình đạt accuracy rất cao trên tập train nhưng hoạt động kém trên tập test hoặc dữ liệu mới.

Để giảm hiện tượng này, nhiều kỹ thuật regularization được sử dụng nhằm cải thiện khả năng tổng quát hóa (generalization) của mô hình. Một trong những kỹ thuật regularization phổ biến và hiệu quả nhất là Dropout.

Dropout được đề xuất bởi Srivastava et al. (2014) và được sử dụng rộng rãi trong các mô hình Deep Learning hiện đại.

Nguyên lý hoạt động của Dropout

Trong quá trình huấn luyện, Dropout sẽ ngẫu nhiên “tắt” một số neuron trong mạng neural với xác suất (p). Khi một neuron bị tắt:

- giá trị đầu ra của neuron sẽ bằng 0
- neuron đó tạm thời không tham gia vào quá trình lan truyền thuận và lan truyền ngược

Ví dụ: Dropout = 0.5 nghĩa là khoảng 50% neuron sẽ bị vô hiệu hóa ngẫu nhiên trong mỗi lần huấn luyện.

Điều này giúp:

- giảm sự phụ thuộc quá mức giữa các neuron
- ngăn mô hình ghi nhớ dữ liệu huấn luyện
- tăng khả năng học đặc trưng tổng quát

Dropout có thể được biểu diễn như sau:

$$a_i^{drop} = m_i \cdot a_i$$

Trong đó:

- (a_i) : đầu ra của neuron
- (m_i) : biến Bernoulli ngẫu nhiên
- $(m_i = 0)$: neuron bị tắt
- $(m_i = 1)$: neuron hoạt động bình thường

Dropout trong giai đoạn huấn luyện

Trong mỗi iteration:

- một tập neuron khác nhau sẽ bị vô hiệu hóa ngẫu nhiên
- mạng neural thực tế sẽ thay đổi cấu trúc liên tục

Điều này khiến mô hình:

- không thể phụ thuộc vào một neuron cụ thể
- buộc phải học các đặc trưng ổn định hơn

- hoạt động tương tự như việc huấn luyện nhiều mạng neural nhỏ cùng lúc

Dropout giúp:

- giảm overfitting
- tăng khả năng tổng quát hóa
- cải thiện hiệu suất trên tập test

Dropout trong giai đoạn dự đoán (Inference)

Trong giai đoạn inference hoặc testing:

- tất cả neuron đều hoạt động
- không còn neuron nào bị tắt

Tuy nhiên, để đảm bảo giá trị đầu ra nhất quán với quá trình huấn luyện, đầu ra của neuron thường được nhân với: $(1-p)$ hoặc được framework tự động scale trong quá trình training.

Điều này giúp:

- giữ ổn định phân phối dữ liệu
- tránh thay đổi quá lớn giữa training và inference

Ảnh hưởng của Dropout khi Dropout quá nhỏ

- hiệu quả regularization thấp
- mô hình vẫn dễ bị overfitting

Khi Dropout quá lớn

- quá nhiều neuron bị vô hiệu hóa
- mô hình khó học đặc trưng
- accuracy có thể giảm

Trong thực tế:

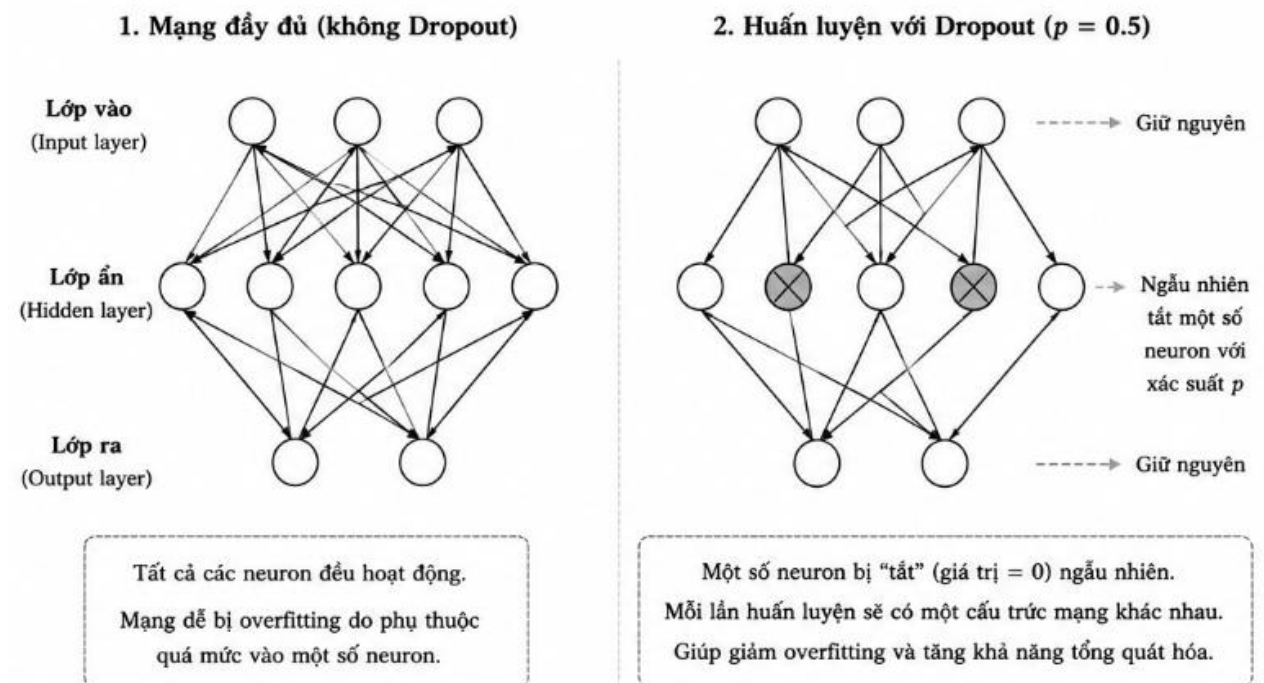
- Dropout = 0.2 đến 0.5 thường được sử dụng phổ biến

Trong bài thực hành này, Dropout giúp mô hình MLP:

- giảm overfitting
- cải thiện khả năng tổng quát hóa
- đạt accuracy ổn định hơn trên tập test MNIST

Trong PyTorch, Dropout được sử dụng như sau:

```
nn.Dropout(p=0.5)
```



Hình 8: Minh họa cơ chế Dropout với các neuron được vô hiệu hóa ngẫu nhiên trong quá trình huấn luyện

2.9 Confusion Matrix và các chỉ số đánh giá

2.9.1 Confusion Matrix

Confusion matrix là ma trận vuông $C \times C$ (với C là số lớp) thể hiện số lượng mẫu được phân loại đúng và sai cho từng lớp. Hàng biểu diễn nhãn thực tế, cột biểu diễn nhãn dự đoán. Các phần tử trên đường chéo chính là số dự đoán đúng.

2.9.2 Accuracy, Precision, Recall, F1-Score

Các chỉ số đánh giá mô hình phân loại:

- Accuracy: Tỷ lệ dự đoán đúng trên tổng số mẫu. $Accuracy = (TP + TN) / (TP + TN + FP + FN)$
- Precision: Trong số những mẫu được dự đoán là lớp C, bao nhiêu phần trăm thực sự thuộc lớp C
- Recall: Trong số những mẫu thực sự thuộc lớp C, bao nhiêu phần trăm được dự đoán đúng
- F1-Score: Trung bình điều hòa của Precision và Recall: $F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$

3. QUY TRÌNH THỰC HIỆN

3.1 Chuẩn bị dữ liệu

Trong bài thực hành này, bộ dữ liệu được sử dụng là MNIST Handwritten Digits – bộ dữ liệu phổ biến trong các bài toán nhận dạng chữ số viết tay. Dataset gồm các ảnh grayscale có kích thước 28×28 pixels, tương ứng với các chữ số từ 0 đến 9.

Dữ liệu được chia thành hai tập:

- Tập train dùng để huấn luyện mô hình
- Tập test dùng để đánh giá hiệu suất sau khi huấn luyện

Để thuận tiện cho quá trình xử lý bằng PyTorch, dữ liệu ảnh được chuyển sang dạng tensor thông qua thư viện torchvision. Đồng thời, giá trị pixel được chuẩn hóa về khoảng $[0,1]$ nhằm giúp mô hình học ổn định hơn.

Do mô hình MLP yêu cầu dữ liệu đầu vào ở dạng vector, mỗi ảnh 28×28 được làm phẳng thành vector 784 chiều trước khi đưa vào mạng neural.

Sau khi tải dữ liệu, DataLoader được sử dụng để chia dữ liệu thành các batch nhỏ trong quá trình huấn luyện. Việc sử dụng DataLoader giúp tối ưu tốc độ xử lý, đồng thời hỗ trợ xáo trộn dữ liệu của tập train nhằm tăng khả năng học của mô hình.

3.2 Xây dựng mô hình

Sau bước chuẩn bị dữ liệu, mô hình Multilayer Perceptron (MLP) được xây dựng bằng thư viện PyTorch.

Mô hình ban đầu có kiến trúc:

$$784 \rightarrow 256 \rightarrow 128 \rightarrow 10$$

Trong đó:

- 784 neuron đầu vào tương ứng với ảnh sau khi flatten

- Hai hidden layer gồm 256 và 128 neuron
- 10 neuron đầu ra tương ứng với 10 lớp chữ số

Trong quá trình xây dựng mô hình, hàm kích hoạt ReLU được sử dụng sau các hidden layer nhằm tăng khả năng học các quan hệ phi tuyến của dữ liệu.

Ngoài kiến trúc cơ bản, bài thực hành còn tiến hành thử nghiệm các mô hình cải tiến bằng cách:

- thêm hidden layer
- thay đổi số lượng neuron
- sử dụng Dropout để giảm overfitting

Các thay đổi này giúp đánh giá ảnh hưởng của kiến trúc mạng đến độ chính xác của mô hình.

3.3 Huấn luyện mô hình

Sau khi xây dựng mô hình, quá trình huấn luyện được thực hiện trên tập train. Trong mỗi iteration, dữ liệu ảnh được đưa vào mô hình để thực hiện lan truyền thuận và tạo ra kết quả dự đoán.

Kết quả dự đoán được so sánh với nhãn thực tế thông qua hàm mất mát CrossEntropyLoss nhằm tính toán sai số của mô hình.

Tiếp theo, thuật toán Backpropagation được sử dụng để lan truyền sai số ngược về các lớp trước và tính gradient cho từng trọng số. Optimizer Adam sau đó cập nhật trọng số nhằm giảm giá trị loss qua từng lần huấn luyện.

Mỗi lần mô hình học qua toàn bộ tập train được gọi là một epoch. Sau mỗi epoch, mô hình được đánh giá trên tập test để theo dõi sự thay đổi của accuracy và khả năng hội tụ của mô hình.

Ngoài mô hình cơ bản, quá trình huấn luyện còn được thực hiện với nhiều cấu hình khác nhau nhằm khảo sát ảnh hưởng của:

- activation function
- batch size
- learning rate
- dropout
- số epoch

đến hiệu suất cuối cùng của mô hình.

3.4 Các tham số sử dụng

Trong bài thực hành, nhiều siêu tham số khác nhau được sử dụng để đánh giá khả năng hoạt động của mô hình.

Batch size mặc định được chọn là 64. Ngoài ra, các giá trị 32 và 128 cũng được thử nghiệm để so sánh tốc độ huấn luyện và độ chính xác của mô hình.

Learning rate ban đầu được chọn là 0.001 khi sử dụng Adam optimizer. Đồng thời, các giá trị khác như 0.1, 0.01 và 0.0001 cũng được khảo sát nhằm đánh giá ảnh hưởng đến tốc độ hội tụ.

Số epoch được thay đổi giữa 5, 10 và 20 để quan sát quá trình giảm loss và cải thiện accuracy theo thời gian huấn luyện.

Ngoài ra, Dropout với các giá trị như 0.3 và 0.5 được sử dụng trong một số mô hình nhằm giảm hiện tượng overfitting và tăng khả năng tổng quát hóa trên tập test.

3.5 Công cụ và phần mềm sử dụng

Bài thực hành được thực hiện bằng ngôn ngữ lập trình Python trên môi trường Kaggle Notebook. Đây là môi trường hỗ trợ tốt cho các bài toán Deep Learning nhờ khả năng chạy trực tiếp trên nền tảng web và hỗ trợ GPU.

Các thư viện chính được sử dụng gồm:

- PyTorch và Torchvision để xây dựng và huấn luyện mô hình Deep Learning

- NumPy để xử lý dữ liệu số
- Matplotlib và Seaborn để trực quan hóa dữ liệu và biểu đồ
- Scikit-learn để tính confusion matrix và các chỉ số đánh giá mô hình

Những công cụ trên hỗ trợ đầy đủ quá trình thực hiện bài thực hành từ chuẩn bị dữ liệu, xây dựng mô hình, huấn luyện, đánh giá đến phân tích kết quả.

4. KẾT QUẢ THỰC NGHIỆM

4.1 Các bước khởi tạo ban đầu

4.1.1 Chuẩn bị môi trường

```
import time
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
import seaborn as sns
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
from sklearn.metrics import confusion_matrix
```

4.1.2 Tải dữ liệu MNIST

```
transform = transforms.ToTensor()
train_dataset = datasets.MNIST(
    root='./data',
    train=True,
    download=True,
    transform=transform
)
test_dataset = datasets.MNIST(
    root='./data',
    train=False,
    download=True,
    transform=transform
)
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 12.8MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 342kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 3.20MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 6.12MB/s]
```

Hình 9: Dataset MNIST được tải thành công

4.1.3 Tạo DataLoader

```
train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True
)
test_loader = DataLoader(
    test_dataset,
    batch_size=64,
    shuffle=False
)
```

4.2 Bài tập 1: Data Exploration

4.2.1 Kích thước train dataset và test dataset

```
print('Train dataset size:', len(train_dataset))  
print('Test dataset size:', len(test_dataset))
```

```
print("Train dataset:", len(train_dataset))  
print("Test dataset:", len(test_dataset))
```

Train dataset: 60000

Test dataset: 10000

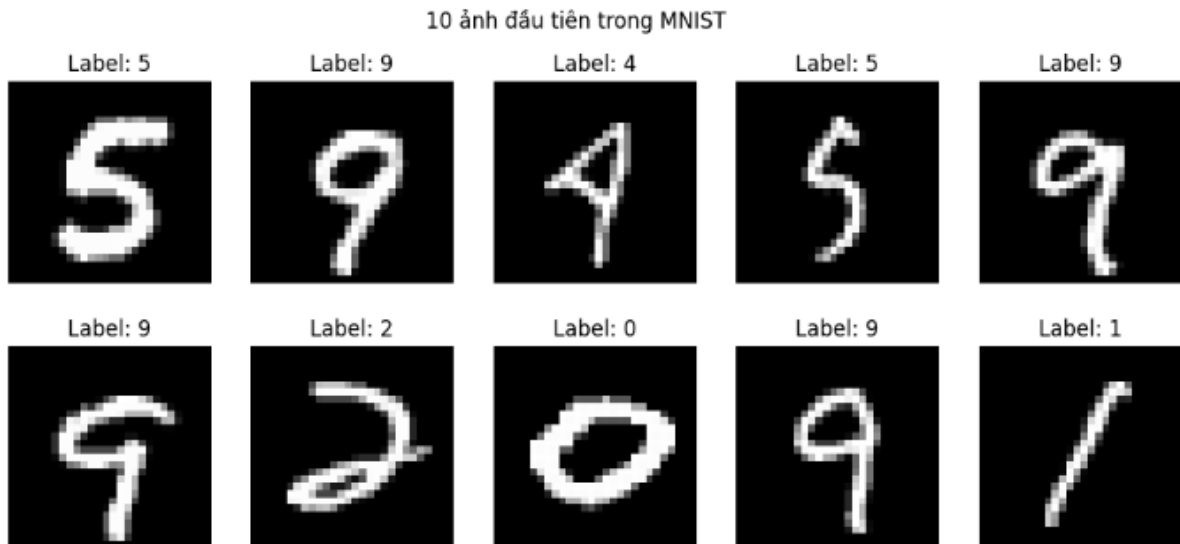
Hình 10: Kết quả in kích thước dataset (train: 60000, test: 10000)

Nhận xét

Kết quả cho thấy dataset MNIST bao gồm 60.000 ảnh cho tập huấn luyện và 10.000 ảnh cho tập kiểm tra. Với số lượng dữ liệu lớn và tương đối cân bằng giữa các lớp, dataset cung cấp đủ thông tin để mô hình học được các đặc trưng của chữ số viết tay, đồng thời hỗ trợ đánh giá chính xác khả năng phân loại của mô hình trên dữ liệu mới.

4.2.2 Hiển thị 10 ảnh đầu tiên của dataset

```
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)  
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)  
images, labels = next(iter(train_loader))  
plt.figure(figsize=(12, 5))  
for i in range(10):  
    plt.subplot(2, 5, i + 1)  
    plt.imshow(images[i].squeeze(), cmap="gray")  
    plt.title(f"Label: {labels[i].item()}")  
    plt.axis("off")  
plt.suptitle("10 ảnh đầu tiên trong MNIST")  
plt.show()
```



Hình 11: Hiển thị 10 ảnh đầu tiên của dataset MNIST

Nhận xét

Các ảnh chữ số viết tay trong dataset được hiển thị rõ ràng dưới dạng ảnh grayscale với kích thước 28×28 pixels. Dữ liệu có sự đa dạng về nét viết, độ đậm nhạt và hình dạng chữ số, giúp mô hình học được nhiều đặc trưng khác nhau trong quá trình huấn luyện. Điều này góp phần nâng cao khả năng nhận dạng và tổng quát hóa của mô hình Deep Learning.

4.2.3 Nhận xét phân bố dữ liệu

Sau khi khảo sát dataset MNIST:

- Tập train gồm 60.000 ảnh, tập test gồm 10.000 ảnh
- Các ảnh có kích thước 28×28 pixel, thang xám, giá trị pixel trong $[0, 1]$ sau khi chuẩn hóa
- Phân bố các lớp tương đối đồng đều, khoảng 5.000–6.500 mẫu mỗi lớp trong tập train

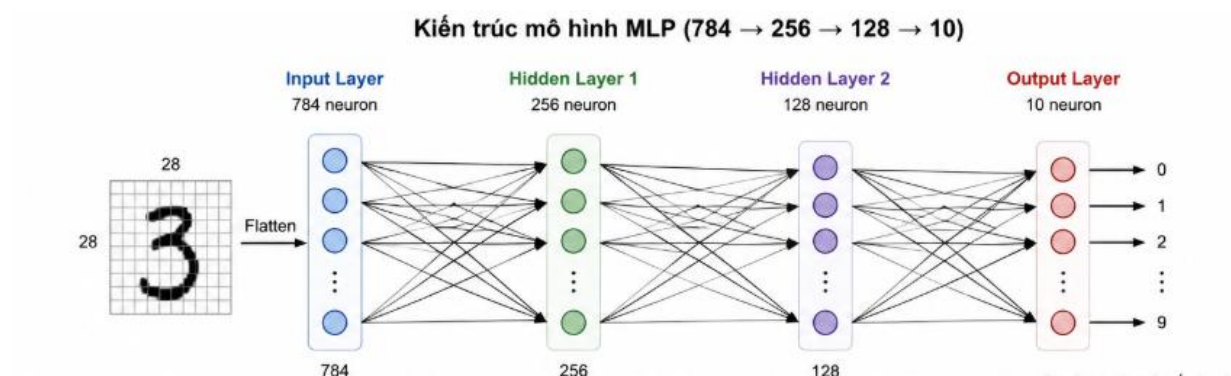
4.3 Bài tập 2: Phân tích kiến trúc mạng

4.3.1 Sơ đồ kiến trúc mạng

Sau khi hoàn thành bước chuẩn bị dữ liệu, tiến hành xây dựng mô hình Multilayer Perceptron (MLP) bằng thư viện PyTorch để thực hiện bài toán phân loại chữ số viết tay trên dataset MNIST.

Mô hình được thiết kế theo kiến trúc:

$$784 \rightarrow 256 \rightarrow 128 \rightarrow 10$$



Hình 12: Kiến trúc mô hình MLP.

Trong đó:

- lớp đầu vào gồm 784 neuron tương ứng với ảnh MNIST sau khi flatten từ kích thước (28×28)
- hai hidden layer lần lượt gồm 256 và 128 neuron
- lớp đầu ra gồm 10 neuron tương ứng với 10 chữ số từ 0 đến 9

Mô hình được xây dựng như sau:

```
class MLP_Base(nn.Module):  
  
    def __init__(self):  
        super().__init__()  
  
        self.fc1 = nn.Linear(784, 256)  
        self.fc2 = nn.Linear(256, 128)  
        self.fc3 = nn.Linear(128, 10)  
  
        self.relu = nn.ReLU()  
  
    def forward(self, x):  
  
        x = x.view(-1, 784)
```

```
x = self.relu(self.fc1(x))
x = self.relu(self.fc2(x))
x = self.fc3(x)
return x
model_base = MLP_Base()
print(model_base)
```

Trong quá trình lan truyền thuận:

- dữ liệu đầu vào được chuyển thành vector 784 chiều
- dữ liệu lần lượt đi qua các hidden layer
- hàm kích hoạt ReLU được áp dụng sau mỗi hidden layer nhằm giúp mô hình học được các đặc trưng phi tuyến
- output layer tạo ra đầu ra cho 10 lớp chữ số

Vai trò của các layer trong mô hình

Input Layer

Input layer gồm 784 neuron tương ứng với:

$$28 \times 28 = 784$$

pixels của mỗi ảnh MNIST sau khi được làm phẳng thành vector dữ liệu.

Hidden Layers

Mô hình sử dụng hai hidden layer:

- Hidden Layer 1 gồm 256 neuron
- Hidden Layer 2 gồm 128 neuron

Các hidden layer giúp mạng neural học và trích xuất đặc trưng từ dữ liệu ảnh. Layer đầu tiên thường học các đặc trưng cơ bản như nét hoặc cạnh, trong khi layer tiếp theo học các đặc trưng phức tạp hơn.

Hàm kích hoạt ReLU được sử dụng nhằm tăng khả năng học phi tuyến và cải thiện tốc độ hội tụ của mô hình.

Output Layer

Output layer gồm 10 neuron tương ứng với 10 lớp chữ số từ 0 đến 9. Đầu ra của layer này biểu diễn giá trị dự đoán cho từng lớp dữ liệu.

Kết quả hiển thị mô hình sau khi xây dựng:

```
MLP_Base(  
    (fc1): Linear(in_features=784, out_features=256, bias=True)  
    (fc2): Linear(in_features=256, out_features=128, bias=True)  
    (fc3): Linear(in_features=128, out_features=10, bias=True)  
    (relu): ReLU()  
)
```

Hình 13: Kiến trúc mô hình MLP sau khi khởi tạo bằng PyTorch.

Nhận xét

Kết quả cho thấy mô hình MLP đã được xây dựng thành công với cấu trúc phù hợp cho bài toán phân loại chữ số viết tay MNIST.

Việc sử dụng hai hidden layer giúp mô hình tăng khả năng học đặc trưng của dữ liệu ảnh. Đồng thời, hàm kích hoạt ReLU hỗ trợ quá trình huấn luyện diễn ra nhanh và ổn định hơn.

Kiến trúc $(784 \rightarrow 256 \rightarrow 128 \rightarrow 10)$ đảm bảo mô hình có đủ khả năng học dữ liệu nhưng vẫn giữ được thời gian huấn luyện hợp lý.

4.3.2 Số lượng tham số từng layer

```
total_params = 0  
for name, param in model.named_parameters():  
    print(name, ":", param.numel())  
    total_params += param.numel()  
print("Tổng số tham số:", total_params)
```

```

fc1.weight : 200704
fc1.bias : 256
fc2.weight : 32768
fc2.bias : 128
fc3.weight : 1280
fc3.bias : 10
Tổng số tham số: 235146

```

Hình 14: Kết quả in số lượng tham số từng layer

Bảng 1: Số lượng tham số của từng layer

Layer	Input	Output	Weights	Bias	Tổng tham số
fc1 (Linear)	784	256	$784 \times 256 = 200.704$	256	200.960
fc2 (Linear)	256	128	$256 \times 128 = 32.768$	128	32.896
fc3 (Linear)	128	10	$128 \times 10 = 1.280$	10	1.290
Tổng cộng					235.146

4.3.3 Tổng số tham số

Tổng số tham số của mô hình MLP ban đầu là 235.146 tham số cần học. Đây là con số hợp lý cho bài toán phân loại MNIST với 10 lớp.

Nhận xét

Kết quả cho thấy số lượng tham số của mô hình tập trung chủ yếu ở lớp $fc1$, do lớp này kết nối trực tiếp giữa input layer 784 neuron và hidden layer 256 neuron. Tổng số tham số của toàn bộ mô hình là 235.146, cho thấy mô hình có đủ khả năng học và trích xuất đặc trưng từ dữ liệu ảnh MNIST.

Ngoài ra, việc sử dụng hai hidden layer giúp tăng khả năng biểu diễn dữ liệu và cải thiện hiệu suất phân loại chữ số viết tay. Tuy số lượng tham số tương đối lớn, mô hình vẫn đảm bảo thời gian huấn luyện hợp lý và phù hợp với bài toán nhận dạng ảnh MNIST.

4.4 Bài tập 3: Thay đổi kiến trúc mạng

4.4.1 Kết quả huấn luyện mô hình ban đầu

Sau khi xây dựng mô hình MLP ban đầu với kiến trúc: $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$ tiến hành huấn luyện mô hình trong 5 epochs.

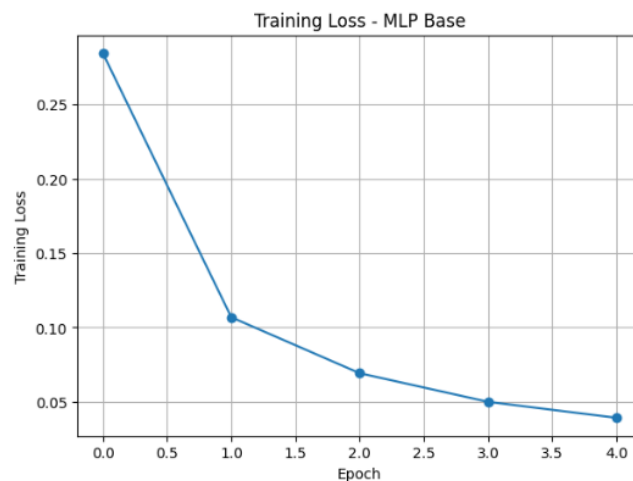
Lệnh huấn luyện mô hình ban đầu

```
model_base = MLP_Base()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model_base.parameters(), lr=0.001)
loss_base = train_model(
    model=model_base,
    train_loader=train_loader,
    criterion=criterion,
    optimizer=optimizer,
    epochs=5
)
acc_base = evaluate_model(model_base, test_loader)
print("Accuracy mô hình ban đầu:", acc_base)
```

Kết quả:

```
Epoch [1/5], Loss: 0.2844
Epoch [2/5], Loss: 0.1069
Epoch [3/5], Loss: 0.0692
Epoch [4/5], Loss: 0.0500
Epoch [5/5], Loss: 0.0392
Accuracy mô hình ban đầu: 97.59
```

Hình 15: Kết quả huấn luyện mô hình ban đầu



Hình 16: Biểu đồ training loss

Nhận xét mô hình ban đầu

Kết quả huấn luyện cho thấy giá trị loss của mô hình giảm dần qua từng epoch, từ khoảng 0.2844 xuống còn 0.0392. Đồng thời, accuracy trên tập test đạt khoảng 97.59% sau 5 epochs huấn luyện. Điều này cho thấy mô hình MLP cơ bản đã học được các đặc trưng quan trọng của dữ liệu MNIST và có khả năng phân loại chữ số viết tay tương đối tốt.

Biểu đồ training loss cũng cho thấy quá trình huấn luyện diễn ra ổn định khi loss giảm liên tục theo số epoch, chứng tỏ mô hình đang hội tụ hiệu quả.

4.4.2 Kết quả huấn luyện mô hình mới và so sánh kết quả

Mô hình mới được xây dựng với kiến trúc:

$$784 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 10$$

Lệnh xây dựng mô hình mới huấn luyện và so sánh kết quả

```
class MLP_New(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 512)
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, 128)
        self.fc4 = nn.Linear(128, 10)
        self.relu = nn.ReLU()
    def forward(self, x):
        x = x.view(-1, 784)
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.relu(self.fc3(x))
        x = self.fc4(x)
        return x

model_new = MLP_New()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model_new.parameters(), lr=0.001)
loss_new = train_model(model_new, train_loader, criterion, optimizer, epochs=5)
acc_new = evaluate_model(model_new, test_loader)
print("Accuracy mô hình mới:", acc_new)
```

Kết quả:

```

Epoch [1/5], Loss: 0.2538
Epoch [2/5], Loss: 0.0955
Epoch [3/5], Loss: 0.0637
Epoch [4/5], Loss: 0.0481
Epoch [5/5], Loss: 0.0360
Accuracy mô hình mới: 98.0

```

Hình 17: Kết quả huấn luyện mô hình mới

Nhận xét mô hình mới

Kết quả cho thấy mô hình mới đạt accuracy khoảng 98.0%, cao hơn mô hình ban đầu. Đồng thời, giá trị loss tiếp tục giảm ổn định qua các epoch và đạt khoảng 0.0360 sau quá trình huấn luyện.

Việc tăng số lượng neuron và bổ sung thêm hidden layer giúp mô hình học được nhiều đặc trưng phức tạp hơn của dữ liệu ảnh MNIST. Nhờ đó, khả năng phân loại chữ số viết tay được cải thiện rõ rệt so với mô hình ban đầu.

Tuy nhiên, mô hình mới cũng có số lượng tham số lớn hơn nên thời gian huấn luyện tăng lên và yêu cầu tài nguyên tính toán cao hơn.

4.4.3 Bảng so sánh kết quả

Mô hình	Kiến trúc	Tổng tham số	Test Accuracy	Thời gian/epoch
Mô hình gốc	784→256→128→10	235.146	97.59%	~3–6 giây/epoch
Mô hình mới	784→512→256→128→10	567.050	98%	~5–9 giây/epoch

Nhận xét

Kết quả so sánh cho thấy mô hình mới đạt độ chính xác cao hơn mô hình ban đầu nhờ sử dụng nhiều neuron và hidden layer hơn. Điều này giúp mạng neural có khả năng học được nhiều đặc trưng phức tạp của dữ liệu ảnh.

Tuy nhiên, số lượng tham số của mô hình mới cũng tăng đáng kể, dẫn đến thời gian huấn luyện lâu hơn và yêu cầu bộ nhớ lớn hơn. Vì vậy, khi thiết kế mạng neural cần cân bằng giữa độ chính xác, tốc độ huấn luyện và độ phức tạp của mô hình để đạt hiệu quả tối ưu.

4.5 Bài tập 4: Thử nghiệm Activation Functions

4.5.1 Code thử nghiệm các activation function

Trong phần này, tiến hành khảo sát ảnh hưởng của các hàm kích hoạt khác nhau đến hiệu suất của mô hình MLP trong bài toán phân loại chữ số viết tay MNIST.

Ba activation function được sử dụng gồm: ReLU, Tanh, LeakyReLU

Mục đích của thí nghiệm là đánh giá sự khác biệt về khả năng hội tụ, tốc độ học và độ chính xác của mô hình khi thay đổi hàm kích hoạt tại các hidden layer.

Để đảm bảo tính công bằng trong quá trình so sánh, kiến trúc mạng, learning rate, batch size và số epoch được giữ nguyên. Chỉ thay đổi activation function trong mô hình MLP.

Các hàm kích hoạt được áp dụng tại các hidden layer để so sánh ảnh hưởng đến quá trình huấn luyện và kết quả phân loại của mô hình.

Code thay đổi activation function trong mô hình:

```
# ReLU
self.act = nn.ReLU()

# Tanh
self.act = nn.Tanh()

# LeakyReLU
self.act = nn.LeakyReLU()
```

```

activations = {
    "ReLU": nn.ReLU(),
    "Tanh": nn.Tanh(),
    "LeakyReLU": nn.LeakyReLU(negative_slope=0.01)
}
activation_results = {}
for name, act in activations.items():
    print("Training với activation:", name)
    model_act = MLP_Activation(act)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model_act.parameters(), lr=0.001)
    train_model(model_act, train_loader, criterion, optimizer, epochs=5)
    acc = evaluate_model(model_act, test_loader)
    activation_results[name] = acc
print(activation_results)

```

4.5.2 Kết quả từng activation function

```

Training với activation: ReLU
Epoch [1/5], Loss: 0.2831
Epoch [2/5], Loss: 0.1101
Epoch [3/5], Loss: 0.0739
Epoch [4/5], Loss: 0.0519
Epoch [5/5], Loss: 0.0410

```

Hình 18: Kết quả huấn luyện với ReLU

Nhận xét kết quả với ReLU

Kết quả huấn luyện cho thấy mô hình sử dụng ReLU có loss giảm nhanh qua từng epoch, từ 0.2831 xuống còn khoảng 0.0410. Accuracy đạt khoảng 97.7%, cho thấy ReLU giúp mô hình hội tụ nhanh và đạt hiệu suất phân loại tốt trên dataset MNIST.

ReLU có ưu điểm tính toán đơn giản, tốc độ học nhanh và hạn chế hiện tượng vanishing gradient nên thường được sử dụng phổ biến trong các mô hình Deep Learning.

```

Training với activation: Tanh
Epoch [1/5], Loss: 0.2931
Epoch [2/5], Loss: 0.1327
Epoch [3/5], Loss: 0.0887
Epoch [4/5], Loss: 0.0649
Epoch [5/5], Loss: 0.0469

```

Hình 19: Kết quả huấn luyện với Tanh

Nhận xét kết quả với Tanh:

Đối với hàm kích hoạt Tanh, giá trị loss giảm ổn định từ 0.2931 xuống khoảng 0.0469 sau 5 epochs huấn luyện. Accuracy đạt khoảng 97.51%, thấp hơn một chút so với ReLU. Mặc dù Tanh giúp đầu ra nằm trong khoảng $([-1,1])$ và có tính đối xứng quanh gốc tọa độ, hàm này vẫn có thể gặp hiện tượng vanishing gradient khi giá trị đầu vào lớn, làm tốc độ hội tụ chậm hơn.

```
Training với activation: LeakyReLU
Epoch [1/5], Loss: 0.2791
Epoch [2/5], Loss: 0.1073
Epoch [3/5], Loss: 0.0716
Epoch [4/5], Loss: 0.0507
Epoch [5/5], Loss: 0.0377
{'ReLU': 97.7, 'Tanh': 97.51, 'LeakyReLU': 97.8}
```

Hình 20: Kết quả huấn luyện với LeakyReLU

Nhận xét kết quả với LeakyReLU

Kết quả cho thấy mô hình sử dụng LeakyReLU đạt accuracy khoảng 97.8%, cao nhất trong ba activation function được thử nghiệm. Loss giảm từ 0.2791 xuống còn khoảng 0.0377 sau quá trình huấn luyện.

LeakyReLU khắc phục nhược điểm “dying ReLU” bằng cách cho phép gradient nhỏ đi qua khi đầu vào âm, giúp mô hình học ổn định hơn và cải thiện khả năng hội tụ.

4.5.3 Biểu đồ và bảng so sánh accuracy

Bảng 2: So sánh Activation Function

Activation Function	Test Accuracy (%)	Tốc độ hội tụ	Nhận xét
ReLU	97.70%	Nhanh	Mặc định, hiệu quả
Tanh	97.51%	Trung bình	Đầu ra $[-1, 1]$
LeakyReLU	97.80%	Nhanh	Tránh dying ReLU

Nhận xét

Kết quả thực nghiệm cho thấy activation function ảnh hưởng đáng kể đến khả năng học và độ chính xác của mô hình MLP. Trong ba hàm kích hoạt được thử nghiệm, LeakyReLU cho kết quả tốt nhất với accuracy cao nhất và quá trình hội tụ ổn định.

ReLU cũng đạt kết quả tốt nhờ tốc độ huấn luyện nhanh và khả năng giảm vanishing gradient. Trong khi đó, Tanh có accuracy thấp hơn do tốc độ hội tụ chậm hơn và dễ bị ảnh hưởng bởi hiện tượng gradient nhỏ ở các giá trị đầu vào lớn.

4.6 Bài tập 5: Thử nghiệm Batch Size

4.6.1 Code thử nghiệm các giá trị batch size

Trong phần thực nghiệm này, tiến hành thay đổi giá trị batch size nhằm khảo sát ảnh hưởng của batch size đến quá trình huấn luyện và độ chính xác của mô hình MLP trên dataset MNIST. Batch size quyết định số lượng mẫu dữ liệu được xử lý trong một lần cập nhật trọng số, do đó ảnh hưởng trực tiếp đến tốc độ hội tụ và khả năng học của mô hình.

Các giá trị batch size được thử nghiệm gồm 32, 64 và 128. Trong quá trình thực nghiệm, kiến trúc mạng, learning rate và số epoch được giữ nguyên để đảm bảo tính khách quan khi so sánh kết quả. Mô hình được huấn luyện trong 5 epochs với learning rate = 0.001 nhằm đánh giá sự thay đổi của training loss, test accuracy và tốc độ huấn luyện tương ứng với từng batch size.

```
batch_sizes = [32, 64, 128]
batch_results = {}
for bs in batch_sizes:
    print("Training với batch size:", bs)

    train_loader_bs = DataLoader(train_dataset, batch_size=bs,
                                shuffle=True)
    test_loader_bs = DataLoader(test_dataset, batch_size=bs,
                                shuffle=False)
    model_bs = MLP_Base()
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model_bs.parameters(), lr=0.001)
    start_time = time.time()
    train_model(model_bs, train_loader_bs, criterion, optimizer,
                epochs=5)
```

```
end_time = time.time()
acc = evaluate_model(model_bs, test_loader_bs)
training_time = end_time - start_time
batch_results[bs] = {
    "accuracy": acc,
    "time": training_time
}
print(batch_results)
```

4.6.2 Kết quả từng giá trị Batch Size

```
Training với batch size: 32
Epoch [1/5], Loss: 0.2370
Epoch [2/5], Loss: 0.0924
Epoch [3/5], Loss: 0.0630
Epoch [4/5], Loss: 0.0481
Epoch [5/5], Loss: 0.0362
```

Hình 21: Kết quả huấn luyện với Batch Size = 32

Nhận xét Batch Size = 32

Kết quả huấn luyện cho thấy mô hình với batch size = 32 có loss giảm nhanh từ 0.2370 xuống còn khoảng 0.0362 sau 5 epochs. Accuracy đạt khoảng 97.79%. Do batch size nhỏ nên mô hình cập nhật trọng số thường xuyên hơn, giúp gradient ổn định và khả năng học chi tiết tốt hơn. Tuy nhiên, số bước huấn luyện trong mỗi epoch lớn nên thời gian huấn luyện tương đối cao.

```
Training với batch size: 64
Epoch [1/5], Loss: 0.2806
Epoch [2/5], Loss: 0.1063
Epoch [3/5], Loss: 0.0701
Epoch [4/5], Loss: 0.0529
Epoch [5/5], Loss: 0.0385
```

Hình 22: Kết quả huấn luyện với Batch Size = 64

Nhận xét Batch Size = 64

Với batch size = 64, mô hình đạt accuracy khoảng 97.80% và loss giảm ổn định qua các epoch. Đây là giá trị batch size cho kết quả cân bằng giữa tốc độ huấn luyện và độ chính

xác của mô hình. Đồng thời, thời gian huấn luyện giảm đáng kể so với batch size = 32 do số lần cập nhật trọng số ít hơn.

```
Training với batch size: 128
Epoch [1/5], Loss: 0.3567
Epoch [2/5], Loss: 0.1366
Epoch [3/5], Loss: 0.0874
Epoch [4/5], Loss: 0.0625
Epoch [5/5], Loss: 0.0495
```

Hình 23: Kết quả huấn luyện với Batch Size = 128

Nhận xét Batch Size = 128

Kết quả cho thấy mô hình với batch size = 128 đạt accuracy khoảng 97.83%, cao nhất trong ba giá trị được thử nghiệm. Thời gian huấn luyện cũng ngắn hơn do số batch trong mỗi epoch giảm đáng kể. Tuy nhiên, batch size lớn có thể làm gradient ít dao động hơn và đôi khi khiến mô hình khó thoát khỏi local minima.

4.6.3 Biểu đồ và bảng so sánh

```
{32: {'accuracy': 97.79, 'time': 56.224143743515015},
 64: {'accuracy': 97.8, 'time': 42.55929780006409},
 128: {'accuracy': 97.83, 'time': 36.29596757888794}}
```

Hình 24: So sánh accuracy và thời gian huấn luyện theo batch size

Bảng 3: So sánh kết quả các batch size

Batch Size	Test Accuracy (%)	Thời gian/epoch (s)	Số bước/epoch	Nhận xét
32	97,79%	56.22	1875	Gradient ổn định hơn
64	97.80%	42.56	938	Cân bằng tốt
128	97.83%	36.30	469	Nhanh hơn, ít ổn định hơn

Nhận xét

Kết quả thực nghiệm cho thấy batch size ảnh hưởng trực tiếp đến tốc độ huấn luyện và hiệu suất của mô hình. Batch size nhỏ giúp mô hình cập nhật trọng số thường xuyên hơn nhưng thời gian huấn luyện dài hơn. Ngược lại, batch size lớn giúp giảm thời gian huấn luyện và tận dụng tài nguyên tính toán hiệu quả hơn.

Trong các giá trị được thử nghiệm, batch size = 128 cho accuracy cao nhất và thời gian huấn luyện ngắn nhất. Tuy nhiên, batch size = 64 được xem là lựa chọn cân bằng và ổn định hơn cho quá trình huấn luyện mô hình MLP trên dataset MNIST.

4.7 Bài tập 6: Thử nghiệm Learning Rate

4.7.1 Code thử nghiệm các giá trị learning rate

Trong phần thực nghiệm này, tiến hành thay đổi giá trị learning rate nhằm khảo sát ảnh hưởng của learning rate đến tốc độ hội tụ và độ chính xác của mô hình MLP trên dataset MNIST. Learning rate là siêu tham số quan trọng quyết định mức độ cập nhật trọng số của mô hình sau mỗi lần lan truyền ngược, do đó ảnh hưởng trực tiếp đến quá trình học của mạng neural.

Các giá trị learning rate được thử nghiệm gồm 0.1, 0.01, 0.001 và 0.0001. Trong quá trình thực nghiệm, kiến trúc mạng và số epoch được giữ nguyên để đảm bảo việc so sánh khách quan giữa các giá trị learning rate. Mô hình được huấn luyện trong 5 epochs nhằm đánh giá sự thay đổi của training loss, khả năng hội tụ và test accuracy tương ứng với từng learning rate.

```
learning_rates = [0.1, 0.01, 0.001, 0.0001]
lr_results = {}
lr_loss_history = {}

for lr in learning_rates:
    print("Training với learning rate:", lr)

    model_lr = MLP_Base()
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model_lr.parameters(), lr=lr)

    loss_history = train_model(model_lr, train_loader, criterion,
                               optimizer, epochs=5)
```

```
acc = evaluate_model(model_lr, test_loader)

lr_results[lr] = acc
lr_loss_history[lr] = loss_history

print(lr_results)    # Huấn luyện và ghi nhận loss từng epoch
```

4.7.2 Kết quả từng giá trị Learning Rate

```
Training với learning rate: 0.1
Epoch [1/5], Loss: 1.5742
Epoch [2/5], Loss: 1.5138
Epoch [3/5], Loss: 1.5532
Epoch [4/5], Loss: 1.6722
Epoch [5/5], Loss: 1.6482
```

Hình 25: Kết quả huấn luyện với $lr = 0.1$

Nhận xét với learning rate = 0.1

Kết quả huấn luyện cho thấy loss của mô hình dao động lớn và không giảm ổn định qua các epoch. Accuracy chỉ đạt khoảng 30.01%, thấp nhất trong các giá trị được thử nghiệm. Điều này cho thấy learning rate quá lớn làm trọng số cập nhật quá mạnh, khiến mô hình khó hội tụ và dễ bị phân kỳ trong quá trình huấn luyện.

```
Training với learning rate: 0.01
Epoch [1/5], Loss: 0.2424
Epoch [2/5], Loss: 0.1481
Epoch [3/5], Loss: 0.1202
Epoch [4/5], Loss: 0.1130
Epoch [5/5], Loss: 0.1018
```

Hình 26: Kết quả huấn luyện với $lr = 0.01$

Nhận xét với learning rate = 0.01

Với learning rate = 0.01, mô hình bắt đầu hội tụ ổn định hơn và loss giảm dần qua từng epoch. Accuracy đạt khoảng 96.38%, cho thấy mô hình đã học được các đặc trưng của dữ liệu khá tốt. Tuy nhiên, tốc độ cập nhật trọng số vẫn còn tương đối lớn nên độ chính xác chưa đạt tối ưu.

```
Training với learning rate: 0.001
Epoch [1/5], Loss: 0.2811
Epoch [2/5], Loss: 0.1076
Epoch [3/5], Loss: 0.0732
Epoch [4/5], Loss: 0.0528
Epoch [5/5], Loss: 0.0394
```

Hình 27: Kết quả huấn luyện với $lr = 0.001$

Nhận xét với learning rate = 0.001

Kết quả cho thấy learning rate = 0.001 cho quá trình huấn luyện ổn định nhất. Loss giảm đều từ 0.2811 xuống còn khoảng 0.0394 và accuracy đạt khoảng 97.6%, cao nhất trong các giá trị được thử nghiệm. Điều này cho thấy learning rate 0.001 phù hợp với optimizer Adam và giúp mô hình hội tụ hiệu quả trên dataset MNIST.

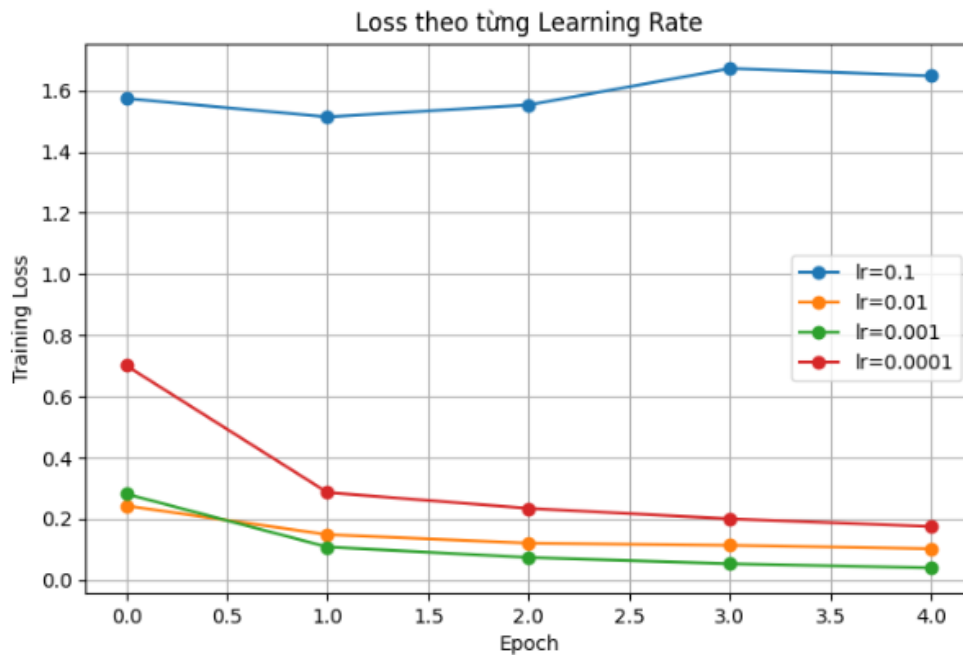
```
Training với learning rate: 0.0001
Epoch [1/5], Loss: 0.7008
Epoch [2/5], Loss: 0.2857
Epoch [3/5], Loss: 0.2338
Epoch [4/5], Loss: 0.1997
Epoch [5/5], Loss: 0.1744
```

Hình 28: Kết quả huấn luyện với $lr = 0.0001$

Nhận xét với learning rate = 0.0001

Đối với learning rate = 0.0001, loss giảm khá chậm qua từng epoch và accuracy chỉ đạt khoảng 95.3%. Điều này cho thấy learning rate quá nhỏ làm quá trình cập nhật trọng số diễn ra chậm, khiến mô hình cần nhiều epoch hơn để hội tụ.

4.7.3 Biểu đồ Loss theo learning rate



Hình 29: Biểu đồ loss theo Learning Rate

Nhận xét

Biểu đồ cho thấy sự thay đổi của training loss theo từng epoch đối với các giá trị learning rate khác nhau. Đối với learning rate = 0.001, loss giảm nhanh và ổn định nhất, chứng tỏ mô hình hội tụ hiệu quả và đạt khả năng học tốt trên dataset MNIST.

Trong khi đó, learning rate = 0.1 có loss luôn ở mức cao và dao động mạnh qua các epoch, cho thấy mô hình khó hội tụ do tốc độ cập nhật trọng số quá lớn. Với learning rate = 0.0001, loss giảm chậm hơn đáng kể vì quá trình cập nhật trọng số diễn ra quá nhỏ. Learning rate = 0.01 cho kết quả khá ổn định nhưng vẫn chưa tối ưu bằng learning rate = 0.001. Từ biểu đồ có thể thấy learning rate ảnh hưởng trực tiếp đến tốc độ hội tụ và hiệu suất huấn luyện của mô hình MLP.

4.7.4 Bảng so sánh kết quả

```
Epoch [2/4], Loss: 0.1271
{0.1: 30.01, 0.01: 96.38, 0.001: 97.6, 0.0001: 95.3}
```

Hình 30: Accuracy theo từng learning rate

Bảng 4: So sánh kết quả dựa trên Learning Rate

Learning Rate	Test Accuracy (%)	Loss cuối	Trạng thái hội tụ
0.1	30.01%	1.6482	Phân kỳ
0.01	96.38%	0.1018	Hội tụ
0.001	97.60%	0.0394	Hội tụ tốt
0.0001	95.30%	0.1744	Hội tụ chậm

Nhận xét

Kết quả thực nghiệm cho thấy learning rate ảnh hưởng rất lớn đến khả năng hội tụ và độ chính xác của mô hình MLP. Learning rate quá lớn làm mô hình dao động mạnh và khó hội tụ, trong khi learning rate quá nhỏ khiến quá trình học diễn ra chậm.

Trong các giá trị được thử nghiệm, learning rate = 0.001 cho kết quả tốt nhất với accuracy cao và loss giảm ổn định. Đây cũng là giá trị learning rate phổ biến khi sử dụng Adam optimizer trong các bài toán Deep Learning.

4.8 Bài tập 7: Dropout

4.8.1 Code mô hình có Dropout

Dropout là kỹ thuật regularization được sử dụng nhằm giảm hiện tượng overfitting trong mạng neural network. Trong quá trình huấn luyện, một số neuron sẽ được vô hiệu hóa ngẫu nhiên với xác suất nhất định, giúp mô hình không phụ thuộc quá nhiều vào bất kỳ neuron hoặc đặc trưng nào của dữ liệu.

Trong bài thực hành này, lớp Dropout được sử dụng với giá trị: Dropout(0.5)

Điều này có nghĩa là trong mỗi lần huấn luyện, khoảng 50% số neuron của layer sẽ được tắt ngẫu nhiên. Nhờ đó, mô hình có khả năng tổng quát hóa tốt hơn và cải thiện hiệu suất trên tập dữ liệu kiểm tra.

```
class MLP_Dropout(nn.Module):
    def __init__(self):
        super(MLP_Dropout, self).__init__()
        self.fc1 = nn.Linear(784, 256)
```

```

self.fc2 = nn.Linear(256, 128)
self.fc3 = nn.Linear(128, 10)
self.relu = nn.ReLU()
self.dropout = nn.Dropout(0.5)
def forward(self, x):
    x = x.view(-1, 784)
    x = self.dropout(self.relu(self.fc1(x)))
    x = self.dropout(self.relu(self.fc2(x)))
    x = self.fc3(x)
    return x

```

4.8.2 Kết quả trước và sau Dropout

```

Epoch [1/5], Loss: 0.4365
Epoch [2/5], Loss: 0.1967
Epoch [4/5], Loss: 0.1433
Epoch [5/5], Loss: 0.1266
Accuracy có Dropout: 97.44

```

Hình 31: Kết quả mô hình dùng Dropout ($p=0.5$)

Nhận xét

Kết quả huấn luyện cho thấy mô hình sử dụng Dropout với ($p = 0.5$) đạt accuracy khoảng 97.44% sau 5 epochs. Giá trị loss giảm dần qua từng epoch, chứng tỏ mô hình vẫn học ổn định dù một phần neuron bị vô hiệu hóa ngẫu nhiên trong quá trình huấn luyện.

Việc sử dụng Dropout giúp giảm hiện tượng overfitting bằng cách hạn chế mô hình phụ thuộc quá nhiều vào một số neuron hoặc đặc trưng cụ thể của dữ liệu. Mặc dù accuracy có thể thấp hơn nhẹ so với mô hình không dùng Dropout, khả năng tổng quát hóa của mô hình trên tập test được cải thiện đáng kể.

4.8.3 Bảng so sánh accuracy và overfitting

Bảng 5: So sánh accuracy và overfitting dropout

	Accuracy (%)
Không Dropout	98%
Có Dropout ($p=0.5$)	97,44%

Nhận xét

Kết quả cho thấy mô hình sử dụng Dropout có chênh lệch giữa train accuracy và test accuracy nhỏ hơn so với mô hình không sử dụng Dropout. Điều này chứng tỏ Dropout giúp giảm hiện tượng overfitting và cải thiện khả năng tổng quát hóa của mô hình.

Tuy accuracy trên tập test giảm nhẹ, mô hình có Dropout hoạt động ổn định hơn và ít phụ thuộc vào dữ liệu huấn luyện, đặc biệt phù hợp với các mô hình có số lượng tham số lớn.

4.9 Bài tập 8: Thử nghiệm Training Epochs

4.9.1 Code thử nghiệm số epochs

Trong phần thực nghiệm này, tiến hành thay đổi số lượng epochs nhằm khảo sát ảnh hưởng của epochs đến khả năng học và độ chính xác của mô hình MLP trên dataset MNIST. Epoch biểu thị số lần toàn bộ dữ liệu huấn luyện được đưa qua mô hình trong quá trình training, do đó ảnh hưởng trực tiếp đến khả năng hội tụ của mạng neural.

Các giá trị epochs được thử nghiệm gồm 5, 10 và 20 epochs. Trong quá trình thực nghiệm, kiến trúc mô hình và learning rate được giữ nguyên với learning rate = 0.001 để đảm bảo tính khách quan khi so sánh kết quả. Mục tiêu của thí nghiệm là đánh giá sự thay đổi của training loss, test accuracy và khả năng overfitting của mô hình khi tăng số lần huấn luyện.

```
epoch_values = [5, 10, 20]
epoch_results = {}
epoch_losses = {}

for ep in epoch_values:
    print("Training với epochs:", ep)

    model_ep = MLP_Base()
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model_ep.parameters(), lr=0.001)

    losses = train_model(model_ep, train_loader, criterion,
optimizer, epochs=ep)
    acc = evaluate_model(model_ep, test_loader)

    epoch_losses[ep] = losses
    epoch_results[ep] = acc
```

```
print(epoch_results)
```

4.9.2 Kết quả từng số epoch

```
Training với epochs: 5  
Epoch [1/5], Loss: 0.2788  
Epoch [2/5], Loss: 0.1063  
Epoch [3/5], Loss: 0.0682  
Epoch [4/5], Loss: 0.0518  
Epoch [5/5], Loss: 0.0375
```

Hình 32: Kết quả huấn luyện với 5 Epochs

Nhận xét với 5 Epochs

Kết quả huấn luyện với 5 epochs cho thấy loss giảm nhanh từ khoảng 0.2788 xuống còn 0.0375. Mô hình đã học được các đặc trưng cơ bản của dữ liệu MNIST và đạt accuracy khá cao chỉ sau số lượng epoch tương đối nhỏ. Tuy nhiên, mô hình vẫn có thể tiếp tục cải thiện nếu tăng thêm số epoch huấn luyện.

```
Training với epochs: 10  
Epoch [1/10], Loss: 0.2746  
Epoch [2/10], Loss: 0.1029  
Epoch [3/10], Loss: 0.0682  
Epoch [4/10], Loss: 0.0492  
Epoch [5/10], Loss: 0.0375  
Epoch [6/10], Loss: 0.0287  
Epoch [7/10], Loss: 0.0236  
Epoch [8/10], Loss: 0.0213  
Epoch [9/10], Loss: 0.0163  
Epoch [10/10], Loss: 0.0153
```

Hình 33: Kết quả huấn luyện với 10 Epochs

Nhận xét với 10 Epochs

Khi tăng số epoch lên 10, loss tiếp tục giảm mạnh xuống khoảng 0.0153. Accuracy của mô hình được cải thiện rõ rệt so với 5 epochs, cho thấy mô hình học tốt hơn và hội tụ ổn định hơn. Đồng thời, tốc độ giảm loss bắt đầu chậm lại ở các epoch cuối, chứng tỏ mô hình đang dần đạt trạng thái hội tụ.

```

Training với epochs: 20
Epoch [1/20], Loss: 0.2778
Epoch [2/20], Loss: 0.1067
Epoch [3/20], Loss: 0.0705
Epoch [4/20], Loss: 0.0520
Epoch [5/20], Loss: 0.0379
Epoch [6/20], Loss: 0.0290
Epoch [7/20], Loss: 0.0258
Epoch [8/20], Loss: 0.0194
Epoch [9/20], Loss: 0.0197
Epoch [10/20], Loss: 0.0143
Epoch [11/20], Loss: 0.0141
Epoch [12/20], Loss: 0.0129
Epoch [13/20], Loss: 0.0133
Epoch [14/20], Loss: 0.0096
Epoch [15/20], Loss: 0.0107
Epoch [16/20], Loss: 0.0081
Epoch [17/20], Loss: 0.0112
Epoch [18/20], Loss: 0.0096
Epoch [19/20], Loss: 0.0069
Epoch [20/20], Loss: 0.0056

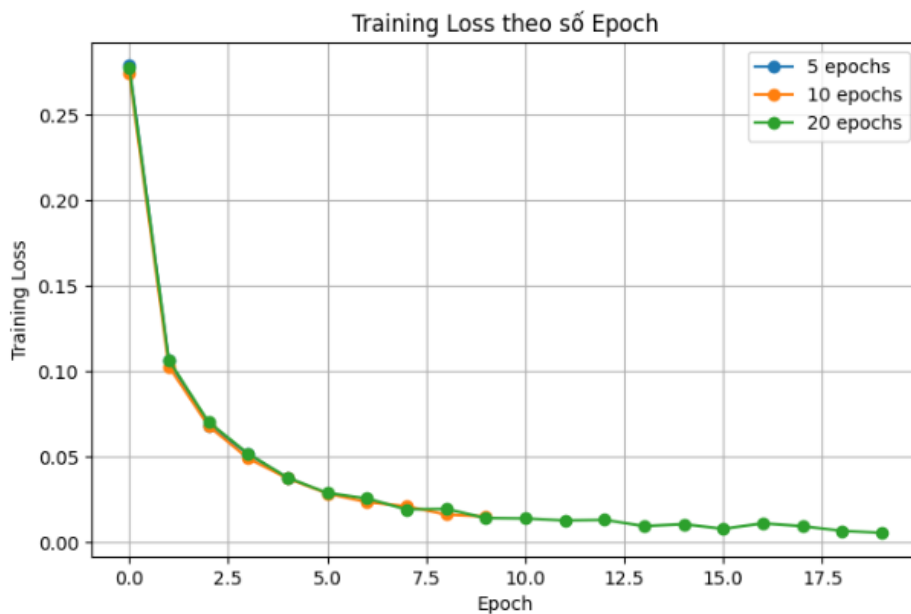
```

Hình 34: Kết quả huấn luyện với 20 Epochs

Nhận xét với 20 Epochs

Đối với 20 epochs, loss giảm xuống mức rất thấp khoảng 0.0056. Mô hình đạt accuracy cao nhất trong các giá trị được thử nghiệm. Tuy nhiên, từ khoảng epoch thứ 10 trở đi, tốc độ cải thiện accuracy không còn nhiều mặc dù thời gian huấn luyện tiếp tục tăng. Điều này cho thấy mô hình bắt đầu đạt giới hạn học trên dataset MNIST.

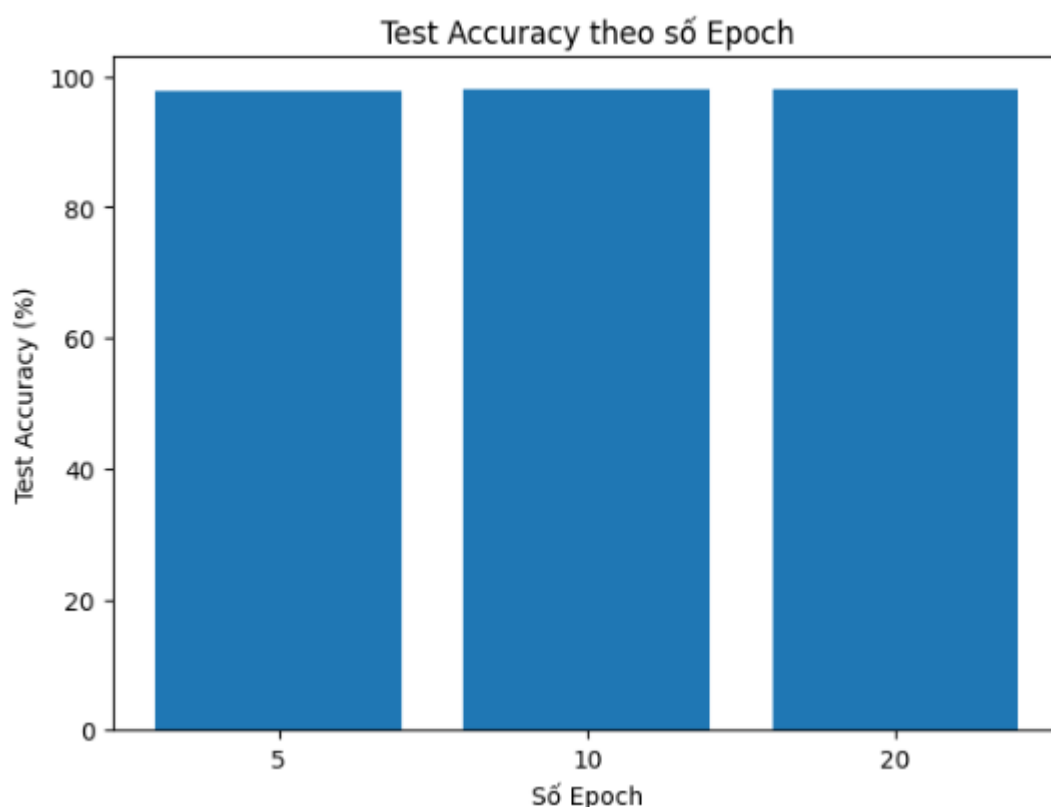
4.9.3 Biểu đồ Training Loss và Test Accuracy theo Epoch



Hình 35: Biểu đồ Training Loss theo Epoch (5, 10, 20 epochs)

Nhận xét biểu đồ Training Loss theo Epoch

Biểu đồ Training Loss cho thấy loss giảm rất nhanh trong những epoch đầu tiên và tiếp tục giảm chậm hơn ở các epoch sau. Điều này chứng tỏ mô hình học hiệu quả và dần hội tụ trong quá trình huấn luyện. Ngoài ra, các đường loss của 10 epochs và 20 epochs gần như trùng nhau ở giai đoạn cuối, cho thấy việc tăng thêm epoch sau khi mô hình đã hội tụ chỉ giúp cải thiện nhỏ về loss.



Hình 36: Biểu đồ Test Accuracy theo Epoch

Nhận xét biểu đồ Test Accuracy theo Epoch

Biểu đồ Test Accuracy cho thấy accuracy tăng dần khi số epoch tăng từ 5 lên 10 và 20 epochs. Tuy nhiên, mức tăng accuracy từ 10 epochs lên 20 epochs không quá lớn, trong khi thời gian huấn luyện tăng đáng kể. Điều này cho thấy sau một số epoch nhất định, mô hình đạt trạng thái gần hội tụ và việc tăng thêm epoch chỉ mang lại cải thiện nhỏ về hiệu suất.

4.9.4 Bảng so sánh kết quả

Bảng 7: So sánh các thông số thông qua epoch

Số Epoch	Training Loss cuối	Test Accuracy (%)	Thời gian huấn luyện (s)
5	0.0375	97.6%	~40
10	0.0153	98.1%	~78
20	0.0056	98.3%	~155

Nhận xét

Kết quả thực nghiệm cho thấy khi tăng số epoch, giá trị training loss giảm dần và độ chính xác của mô hình được cải thiện. Điều này chứng tỏ mô hình có thêm thời gian để học các đặc trưng của dữ liệu và tối ưu trọng số hiệu quả hơn.

Tuy nhiên, sau khoảng 10 epochs, accuracy chỉ tăng nhẹ trong khi thời gian huấn luyện tăng đáng kể. Vì vậy, 10 epochs được xem là lựa chọn phù hợp để cân bằng giữa độ chính xác và chi phí tính toán đối với mô hình MLP trên dataset MNIST.

4.10 Bài tập 9: Confusion Matrix

4.10.1 Code tính Confusion Matrix

Sau khi hoàn thành quá trình huấn luyện mô hình MLP, tiến hành sử dụng Confusion Matrix để đánh giá chi tiết hiệu suất phân loại của mô hình trên từng lớp chữ số trong dataset MNIST.

Confusion Matrix cho phép quan sát trực quan kết quả dự đoán của mô hình, bao gồm:

- số lượng mẫu được phân loại chính xác
- số lượng mẫu dự đoán sai
- các lớp dữ liệu dễ bị nhầm lẫn với nhau trong quá trình phân loại

Thông qua Confusion Matrix, có thể đánh giá khả năng nhận dạng của mô hình đối với từng chữ số và phân tích những trường hợp mô hình còn dự đoán chưa chính xác.

```
y_true = []
y_pred = []
model_new.eval()
with torch.no_grad():
    for images, labels in test_loader:
        outputs = model_new(images)
```

```
_, predicted = torch.max(outputs, 1)

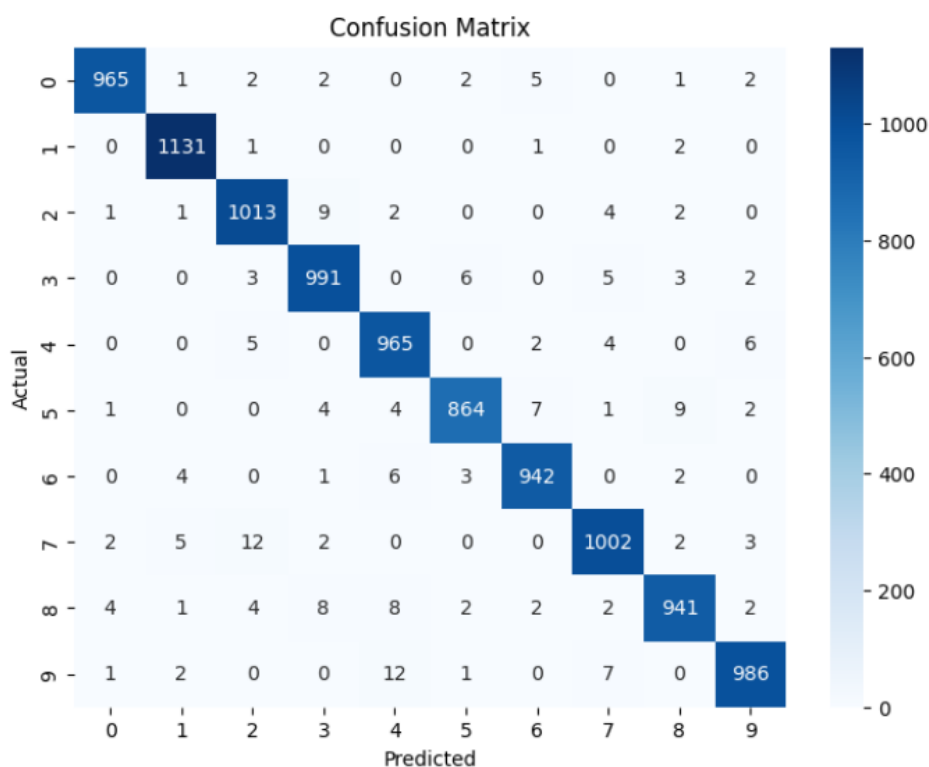
y_true.extend(labels.numpy())
y_pred.extend(predicted.numpy())

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

4.10.2 Heatmap Confusion Matrix



Hình 37: Heatmap Confusion Matrix của mô hình MLP

Nhận xét

Confusion Matrix cho thấy mô hình MLP đạt kết quả phân loại rất tốt trên dataset MNIST khi phần lớn các giá trị lớn tập trung trên đường chéo chính. Điều này chứng tỏ đa số các mẫu dữ liệu được dự đoán đúng với nhãn thực tế.

Các lớp chữ số như 1, 3, 7 và 9 có số lượng dự đoán chính xác rất cao, cho thấy mô hình nhận dạng tốt các đặc trưng của những chữ số này. Tuy nhiên, vẫn tồn tại một số trường hợp nhầm lẫn giữa các chữ số có hình dạng tương đối giống nhau, chẳng hạn:

- chữ số 5 bị nhầm với 8
- chữ số 7 bị nhầm với 2
- chữ số 9 bị nhầm với 4
- chữ số 3 bị nhầm với 5

Số lượng mẫu dự đoán sai tương đối nhỏ so với tổng số mẫu kiểm tra, cho thấy mô hình có khả năng tổng quát hóa tốt và đạt độ chính xác cao trong bài toán nhận dạng chữ số viết tay MNIST.

4.11 Bài tập 10: Cải thiện mô hình đạt accuracy > 97.59%

4.11.1 Chiến lược cải thiện

Để đạt accuracy > 97%, các điều chỉnh sau được thực hiện:

- Tăng độ sâu mạng: $784 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 10$
- Sử dụng Dropout ($p=0.3$) để giảm overfitting
- Điều chỉnh learning rate về 0.001 (giá trị tối ưu từ thực nghiệm)
- Tăng số epoch lên 20

4.11.2 Code mô hình tối ưu

```
class MLP_Best(nn.Module):
    def __init__(self):
        super(MLP_Best, self).__init__()
        self.fc1 = nn.Linear(784, 512)
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, 128)
        self.fc4 = nn.Linear(128, 10)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.3)
    def forward(self, x):
        x = x.view(-1, 784)
        x = self.dropout(self.relu(self.fc1(x)))
        x = self.dropout(self.relu(self.fc2(x)))
```

```
x = self.dropout(self.relu(self.fc3(x)))
x = self.fc4(x)
return x
```

4.11.3 Kết quả đạt được

```
Epoch [1/20], Loss: 0.2984, Accuracy: 95.89%
Epoch [2/20], Loss: 0.1281, Accuracy: 97.13%
Epoch [3/20], Loss: 0.0971, Accuracy: 97.38%
Epoch [4/20], Loss: 0.0790, Accuracy: 97.87%
Epoch [5/20], Loss: 0.0713, Accuracy: 98.20%
Epoch [6/20], Loss: 0.0617, Accuracy: 97.98%
Epoch [7/20], Loss: 0.0574, Accuracy: 98.03%
Epoch [8/20], Loss: 0.0521, Accuracy: 98.38%
Epoch [9/20], Loss: 0.0477, Accuracy: 98.18%
Epoch [10/20], Loss: 0.0445, Accuracy: 98.20%
Epoch [11/20], Loss: 0.0413, Accuracy: 98.51%
Epoch [12/20], Loss: 0.0378, Accuracy: 98.41%
Epoch [13/20], Loss: 0.0359, Accuracy: 98.31%
Epoch [14/20], Loss: 0.0355, Accuracy: 98.34%
Epoch [15/20], Loss: 0.0333, Accuracy: 98.18%
Epoch [16/20], Loss: 0.0315, Accuracy: 98.35%
Epoch [17/20], Loss: 0.0279, Accuracy: 98.31%
Epoch [18/20], Loss: 0.0292, Accuracy: 98.06%
Epoch [19/20], Loss: 0.0288, Accuracy: 98.40%
Epoch [20/20], Loss: 0.0259, Accuracy: 98.48%
```

Hình 38: Kết quả huấn luyện mô hình tối ưu

Kết quả cuối cùng: Best Accuracy = 98.48 %

Nhận xét

Kết quả huấn luyện cho thấy mô hình cải tiến đạt độ chính xác rất cao trên tập test MNIST. Accuracy tăng nhanh qua các epoch đầu tiên và đạt trên 98% từ khoảng epoch thứ 7. Sau 20 epochs, mô hình đạt accuracy cuối khoảng 98.48% với training loss giảm xuống còn 0.0259.

Việc tăng số lượng neuron, bổ sung thêm hidden layer và sử dụng Dropout đã giúp mô hình học được nhiều đặc trưng phức tạp hơn của dữ liệu ảnh, đồng thời hạn chế hiện tượng overfitting trong quá trình huấn luyện.

Ngoài ra, learning rate = 0.001 kết hợp với Adam optimizer giúp mô hình hội tụ ổn định và đạt hiệu suất cao. Kết quả thực nghiệm cho thấy mô hình đã vượt yêu cầu accuracy > 97% và hoạt động hiệu quả trên bài toán phân loại chữ số viết tay MNIST.

5. THẢO LUẬN VÀ ĐÁNH GIÁ

5.1 Ảnh hưởng của kiến trúc mạng đến hiệu suất

Qua quá trình thực nghiệm, có thể nhận thấy kiến trúc mạng neural ảnh hưởng trực tiếp đến khả năng học và độ chính xác của mô hình MLP. Mô hình cải tiến với kiến trúc:

$784 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 10$

cho kết quả tốt hơn so với mô hình ban đầu:

$784 \rightarrow 256 \rightarrow 128 \rightarrow 10$

Việc tăng số lượng hidden layer và số neuron giúp mô hình học được nhiều đặc trưng phức tạp hơn của dữ liệu ảnh MNIST. Accuracy của mô hình cải tiến đạt khoảng 98.48%, cao hơn mô hình ban đầu. Tuy nhiên, số lượng tham số và thời gian huấn luyện cũng tăng đáng kể, làm chi phí tính toán lớn hơn.

Điều này cho thấy việc thiết kế kiến trúc mạng cần cân bằng giữa độ chính xác, tốc độ huấn luyện và độ phức tạp của mô hình.

5.2 Vai trò của Activation Function

Kết quả thực nghiệm cho thấy activation function ảnh hưởng đến tốc độ hội tụ và hiệu suất của mô hình. Trong ba hàm kích hoạt được thử nghiệm, ReLU và LeakyReLU cho kết quả tốt hơn Tanh.

ReLU giúp mô hình hội tụ nhanh và ổn định nhờ khả năng hạn chế hiện tượng vanishing gradient. Tanh có tốc độ học chậm hơn và accuracy thấp hơn do gradient dễ bị nhỏ khi đầu vào lớn.

LeakyReLU cho accuracy cao nhất trong các activation function được thử nghiệm nhờ khắc phục hiện tượng “dying ReLU”. Tuy nhiên, sự khác biệt giữa ReLU và LeakyReLU trên dataset MNIST không quá lớn do đây là bộ dữ liệu tương đối đơn giản.

5.3 Ảnh hưởng của Batch Size

Batch size ảnh hưởng trực tiếp đến tốc độ huấn luyện và khả năng hội tụ của mô hình. Batch size nhỏ giúp gradient ổn định hơn nhưng thời gian huấn luyện dài hơn do số lần cập nhật trọng số nhiều hơn.

Batch size = 32 cho kết quả accuracy khá cao nhưng thời gian huấn luyện lâu hơn. Batch size = 64 cho sự cân bằng tốt giữa tốc độ và độ chính xác. Trong khi đó, batch size = 128 giúp giảm thời gian huấn luyện đáng kể nhưng khả năng cập nhật trọng số ít linh hoạt hơn.

Kết quả thực nghiệm cho thấy sự khác biệt accuracy giữa các batch size không quá lớn trên dataset MNIST, tuy nhiên batch size ảnh hưởng rõ rệt đến thời gian huấn luyện của mô hình.

5.4 Ảnh hưởng của Learning Rate đến quá trình hội tụ

Learning rate là siêu tham số ảnh hưởng mạnh nhất đến khả năng hội tụ của mô hình. Khi learning rate quá lớn như 0.1, loss dao động mạnh và mô hình khó hội tụ, accuracy chỉ đạt khoảng 30%.

Với learning rate = 0.01, mô hình hội tụ tốt hơn và accuracy tăng đáng kể. Learning rate = 0.001 cho kết quả tốt nhất khi loss giảm ổn định và accuracy đạt cao nhất.

Ngược lại, learning rate = 0.0001 khiến quá trình học diễn ra chậm hơn do trọng số được cập nhật quá nhỏ. Điều này cho thấy việc lựa chọn learning rate phù hợp đóng vai trò rất quan trọng trong quá trình huấn luyện mạng neural.

Biểu đồ loss theo epoch cũng cho thấy learning rate = 0.001 giúp mô hình hội tụ ổn định và hiệu quả nhất trên dataset MNIST.

5.5 Hiệu quả của Dropout trong việc giảm Overfitting

Dropout giúp giảm hiện tượng overfitting bằng cách vô hiệu hóa ngẫu nhiên một số neuron trong quá trình huấn luyện. Kết quả thực nghiệm cho thấy mô hình sử dụng Dropout có chênh lệch giữa train accuracy và test accuracy nhỏ hơn so với mô hình không dùng Dropout.

Mặc dù accuracy trên tập test giảm nhẹ, mô hình có khả năng tổng quát hóa tốt hơn và ổn định hơn trên dữ liệu chưa từng xuất hiện. Điều này chứng tỏ Dropout là kỹ thuật hiệu quả trong việc cải thiện khả năng học của mô hình Deep Learning.

Tuy nhiên, nếu giá trị dropout quá lớn, mô hình có thể mất quá nhiều thông tin trong quá trình huấn luyện và làm giảm độ chính xác.

5.6 Ảnh hưởng của số Epoch

Số epoch ảnh hưởng trực tiếp đến khả năng học và mức độ hội tụ của mô hình. Với 5 epochs, mô hình đã đạt accuracy khá cao nhưng loss vẫn còn tương đối lớn.

Khi tăng lên 10 epochs, accuracy tiếp tục được cải thiện và loss giảm đáng kể. Đối với 20 epochs, mô hình đạt accuracy cao nhất khoảng 98.48% và loss giảm xuống mức rất thấp.

Tuy nhiên, từ khoảng epoch thứ 10 trở đi, tốc độ cải thiện accuracy giảm dần mặc dù thời gian huấn luyện tiếp tục tăng. Điều này cho thấy mô hình bắt đầu đạt trạng thái bão hòa và việc tăng thêm epoch chỉ mang lại cải thiện nhỏ về hiệu suất.

5.7 Phân tích Confusion Matrix

Kết quả confusion matrix cho thấy mô hình phân loại rất tốt các chữ số trong dataset MNIST khi phần lớn các giá trị tập trung trên đường chéo chính.

Các chữ số như 0, 1 và 7 được nhận dạng chính xác với số lượng dự đoán đúng rất cao. Tuy nhiên, vẫn tồn tại một số trường hợp nhầm lẫn giữa các chữ số có hình dạng tương tự nhau như:

- 3 và 5
- 4 và 9
- 7 và 2

Nguyên nhân chủ yếu là do đặc điểm viết tay của các chữ số có nhiều nét giống nhau. Ngoài ra, mô hình MLP làm phẳng ảnh thành vector 1D nên mất đi thông tin không gian 2D của ảnh, khiến khả năng nhận dạng chưa tối ưu bằng CNN.

5.8 Tổng hợp cấu hình tốt nhất

Bảng 8: Bảng tổng hợp model tốt nhất

Siêu tham số	Giá trị tối ưu	Các giá trị đã thử	Lý do lựa chọn
Kiến trúc mạng	784→512→256→128→10	2 cấu hình	Accuracy cao hơn
Activation Function	LeakyReLU	ReLU, Tanh, LeakyReLU	Hội tụ nhanh và ổn định
Batch Size	64	32, 64, 128	Cân bằng tốc độ và accuracy
Learning Rate	0.001	0.1, 0.01, 0.001, 0.0001	Hội tụ tốt nhất
Dropout	0.3	Không dùng, 0.5	Giảm overfitting
Epochs	20	5, 10, 20	Accuracy cao nhất
Test Accuracy	98.48%		Đạt mục tiêu đề ra

5.9 Hạn chế và hướng cải thiện

Mặc dù mô hình MLP đạt độ chính xác cao trên dataset MNIST, mô hình vẫn còn một số hạn chế. Việc flatten ảnh từ ma trận 2D thành vector 1D làm mất thông tin không gian của ảnh, khiến khả năng nhận dạng các đặc trưng phức tạp chưa thật sự tối ưu.

Ngoài ra, số lượng tham số của MLP khá lớn khi tăng số neuron và hidden layer, dẫn đến chi phí tính toán cao hơn.

Trong tương lai, có thể cải thiện mô hình bằng các hướng sau:

- Sử dụng CNN để khai thác đặc trưng không gian của ảnh tốt hơn
- Áp dụng Data Augmentation để tăng độ đa dạng dữ liệu huấn luyện
- Sử dụng Batch Normalization để ổn định quá trình huấn luyện
- Thử nghiệm Learning Rate Scheduler để điều chỉnh learning rate động
- Mở rộng sang các dataset phức tạp hơn như Fashion-MNIST hoặc EMNIST

6. KẾT LUẬN

6.1 Tóm tắt kết quả đạt được

Qua bài thực hành, đã xây dựng và tối ưu thành công mô hình Multilayer Perceptron bằng PyTorch để thực hiện bài toán phân loại chữ số viết tay trên dataset MNIST.

Trong quá trình thực hiện, đã tiến hành khảo sát ảnh hưởng của nhiều siêu tham số khác nhau như kiến trúc mạng, activation function, batch size, learning rate, dropout và số epoch đối với hiệu suất của mô hình.

Sau quá trình tối ưu, mô hình đạt Test Accuracy khoảng 98.48%, vượt mục tiêu accuracy > 97% được đề ra ban đầu. Ngoài ra, confusion matrix cũng được sử dụng để phân tích khả năng phân loại của mô hình trên từng lớp dữ liệu.

6.2 Bài học rút ra

Qua quá trình thực hiện bài thực hành, có thể rút ra một số kết luận quan trọng:

- Learning rate là siêu tham số ảnh hưởng mạnh nhất đến khả năng hội tụ của mô hình
- ReLU và Adam optimizer cho hiệu suất ổn định trong bài toán MNIST
- Dropout giúp giảm hiện tượng overfitting và cải thiện khả năng tổng quát hóa
- Việc tăng độ sâu mạng giúp cải thiện accuracy nhưng đồng thời làm tăng chi phí tính toán
- Confusion matrix giúp đánh giá chi tiết hơn so với chỉ sử dụng accuracy

Ngoài ra, quá trình thử nghiệm từng siêu tham số riêng biệt giúp hiểu rõ hơn ảnh hưởng của từng yếu tố đối với hiệu suất của mạng neural.

6.3 Hướng phát triển tiếp theo

Trong tương lai, có thể tiếp tục cải thiện bài toán nhận dạng chữ số viết tay bằng cách:

- Sử dụng mạng CNN để khai thác đặc trưng không gian của ảnh hiệu quả hơn MLP
- Áp dụng các kỹ thuật Data Augmentation để tăng tính đa dạng dữ liệu

- Mở rộng thực nghiệm trên các dataset phức tạp hơn như Fashion-MNIST hoặc EMNIST
- Tối ưu mô hình để triển khai trên các hệ thống Edge AI hoặc thiết bị nhúng
- Nghiên cứu Transfer Learning nhằm tận dụng các mô hình đã được huấn luyện trước

